AI AND DRONE BASED BRIDGE DEFECT DETECTION USING YOLO V8 MODEL

A PROJECT REPORT

Submitted by

PAVIYA R **(730322243018)**

RAGASUDHA S S **(730322243019)**

THAARANI I **(730322243027)**

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KANGEYAM INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

NATHAKADAIYUR – 638 108

MAY 2026

KANGEYAM INSTITUTIONS
KIT
Empowering Minds

KANGEYAM
Institute of Technology
Approved by AICTE, New Delhi | Affiliated to Anna University, Chennai
ISO Certified Institution | Recognised u/s 2(f) of UGC
Accredited by NBA : UG - CIVIL, CSE, ECE, EEE & MECH
EBET Knowledge Park, Nathakadaiyur, Kangeyam,
Tiruppur-638 108, Tamil Nadu, India.
AUTONOMOUS

NAAC A+
ACCREDITED INSTITUTION with 3.36 CGPA on 4.0 scale

NBA
NATIONAL BOARD OF ACCREDITATION

www.kitech.edu.in

BONAFIDE CERTIFICATE

Certified that this project report "AI AND DRONE BASED BRIDGE DEFECT DETECTION USING YOLO V8 MODEL" is the Bonafide work of **P A V I Y A R (730322243018), RAGASUDHA S S (730322243019),** and **T H A A R A N I I (730322243027)** who carried out the project work under my supervision.

SIGNATURE

Dr.S.SELVI B.E.,M.E.,Ph.D.,

HEAD OF THE DEPARTMENT

Department of Artificial
Intelligence and Data Science
Kangeyam Institute of Technology
Nathakadaiyur – 638 108

SIGNATURE

M.NIVETHINI BE.,M.E.,(Ph.D).,

SUPERVISOR

Assistant Professor (Sr.Gr.)
Department of Artificial
Intelligence and Data Science
Kangeyam Institute of Technology
Nathakadaiyur – 638 108

Submitted for the End Semester Viva Voce examination held on_____for **AD3811 – Project Work** during the academic year 2025-2026.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First of all, we thank the almighty God for the blessings and support given to us to complete this project successfully.

We would like to express our deep and heartfelt thanks to our respected Chairman **Thiru. N. RAMALINGAM**, respected and Honourable Secretary **Thiru. C. K. VENKATACHALAM**, beloved and Honourable Correspondent **Thiru. S. ANANDAVADIVEL** and our responsible Treasurer **Thiru C. K. BALASUBRAMANIAM** for giving motivation and providing all necessary facilities for the successful completion of the project.

We take this opportunity to express our profound gratitude and deep regard to **Dr R. V. MAHENDRA GOWDA, B.E., M.E., Ph.D.,** Chief Executive Officer, Kangeyam Institute of technology, for his constant encouragement throughout the course of our work.

With pleasure, we would like to express our gratefulness to **Dr S. RAMKUMAR, B.E., M.E., Ph.D.,** Principal, for the moral support and deeds in bringing out this project successfully.

We are highly obliged to **Dr S. SELVI, B.E., M.E., Ph.D.,** Head of the Department, Department of Artificial Intelligence and Data Science, for her guidance, suggestions and active encouragement for the fruitful completion of the project.

We sincerely thank our Project Coordinator, **Mr V. RAJARAM, M.C.A., M.E., (Ph.D.), and Ms M. NIVETHINI, B.E., M.E., Ph.D.,** Project Guide for encouraging and supporting us throughout the project.

We express our thanks to all our Teaching and Non-Teaching staff members for their advice and encouragement to do the project work with full involvement and enthusiasm. Finally, we would like to thank all our friends and family members for providing us with constant support to complete this project work

ABSTRACT

Ensuring the structural stability of bridges, particularly flyovers, is vital for maintaining safe and reliable transportation infrastructure. Traditional bridge inspection methods are largely manual, requiring significant time, labor, and expertise, while remaining prone to human error. These approaches often overlook minor defects such as cracks, corrosion, and surface degradation in their early stages, reducing the effectiveness of preventive maintenance strategies. Although recent advancements in unmanned aerial vehicles (UAVs) have improved data acquisition by enabling access to hard-to-reach areas, the analysis of collected data is still predominantly performed offline. This delay limits real-time decision-making and reduces the responsiveness of maintenance systems. Furthermore, many existing automated inspection solutions depend on high-end GPUs and advanced computational devices, resulting in high deployment and operational costs that restrict scalability, particularly in resource-constrained environments.

To address these limitations, this work proposes a low-cost, scalable edge-to-cloud automated bridge inspection framework capable of real-time defect detection, severity assessment, and lifespan estimation. The system employs an affordable ESP32-CAM 360 module to continuously capture visual data from bridge surfaces. Instead of performing heavy computations at the edge, the captured data is transmitted to a centralized server, where a YOLO v8-based deep learning model performs multi-class defect classification. The model further evaluates defect severity and predicts the remaining useful life of structural components.This architecture enables continuous monitoring, faster inspection cycles, and cost-effective deployment. Ultimately, the framework bridges the gap between affordability and advanced infrastructure health monitoring, providing a practical solution for large-scale, real-time bridge inspection systems. The accuracy percentage of the **AI and Drone Based Bridge Defect Detection** project is **92–96%** using the **YOLO v8** model.

KEYWORDS: Bridge Defect Detection, YOLO v8, Computer Vision, Internet of Things, Real-Time Monitoring, Structural Health Monitoring

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
	LIST OF ABBREVIATIONS	**ix**
1.	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM STATEMENT	3
	1.3 CATEGORY OF THE PROJECT	4
	1.3.1 Category I: Safety / Sustainability /Cost	4
	1.3.2 Category II: Application / Research	4
	1.3.3 Relevance to Sustainable Development Goals (SDGs)	4
2.	**LITERATURE SURVEY**	**5**
	2.1 INTRODUCTION	5
	2.2 LITERATURE SURVEY	5
	2.2.1 AI and Drone based bridge defect detection Using YOLO v8 model	5
	2.2.2 Defect Classification Using Deep Learning	5
	2.2.3 YOLO v8 for Smart detection And classification	6

	2.2.4 Implementation of AI and Drone based bridge defect detection	6
	2.2.5 Object Classifier with Image Processing and YOLO v8 model	6
3.	**SYSTEM ANALYSIS**	**10**
	3.1 EXISTING SYSTEM	10
	3.2 PROPOSED SYSTEM	11
4.	**SYSTEM SPECIFICATION**	**13**
	4.1 HARDWARE CONFIGURATION	13
	4.2 SOFTWARE CONFIGURATION	13
5.	**SOFTWARE DESCRIPTION**	**14**
	5.1 PYTHON 3.X	14
	5.2 TENSORFLOW & KERAS	14
	5.3 OPENCV	15
	5.4 YOLO V8 MODEL	15
	5.5 SQLITE	15
	5.6 VISUAL STUDIO CODE	16
	5.7 ESP CAM-360	17
6.	**PROJECT DESCRIPTION**	**18**
	6.1 PROBLEM IDENTIFICATION	18
	6.2 LIST OF MODULES	18
	6.3 MODULES DESCRIPTION	18
	6.3.1 Image Acquisition	18
	6.3.2 Image Processing	19
	6.3.3 defect Classification	19
	6.3.4 Database Management	19

6.3.5 User Interface & Output 20

6.3.6 Pseudocode 20

7. SYSTEM IMPLEMENTATION 24

7.1 IMPLEMENTATION 24

7.2 EXPERIMENTAL SETUP 24

7.3 SOFTWARE DEVELOPMENT 24

7.3.1 Embedded System Programming 25

7.3.2 Machine Learning Model Integration 25

7.3.3 Backend and API Development 25

7.3.4 Data Management 26

7.4 MACHINE LEARNING MODEL TRAINING 27

7.4.1 Data Collection 27

7.4.2 Data Preprocessing 28

7.4.3 Model Training (Transfer Learning) 28

7.4.4 Model Selection (MobileNetV2) 28

7.4.5 Model Evaluation 28

7.5 SYSTEM INTEGRATION 29

8. SYSTEM TESTING 32

8.1 INTRODUCTION 32

8.2 UNIT TESTING 32

8.3 HARDWARE-IN-THE-LOOP (HIL) 32

8.4 TEST CASES 33

8.5 RESULTS AND DISCUSSIONS 35

9. CONCLUSION AND FUTURE ENHANCEMENT 39

9.1 CONCLUSION 39

9.2 FUTURE ENHANCEMENT 39

APPENDICES 40

A. SOURCE CODE 40

B. SCREENSHOTS 45

C. PUBLICATIONS 48

REFERENCES 54

LIST OF TABLES

TABLE NO.	NAME OF THE TABLE	PAGE NO.
2.3	Comparison of object detection Models	7
7.1	The Image Processing of YOLO v8 Model	27
7.2	Bridge defect detection of YOLO v8 Model	29
8.1	Check Response on Capturing Image and Classifying defect	33
8.2	Verify Storage of Classification Result in MongoDB	34
8.3	Comparison of Working in Manual, Camera Based and YOLO v8 Based Sorting	35
8.4	Performance Comparison – Existing Vs Proposed System	37

LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURES	PAGE NO.
3.1	Workflow Diagram of Defect classification	13
6.1	Workflow Diagram of Waste Classification Model	23
7.1	Workflow Diagram of YOLO v8 Model	31
8.1	Performance Comparison of Existing and Proposed System	38
B.1	Project Structure	45
B.2	YOLO v8 Model Training	45
B.3	Bridge defect classification	46
B.4	Defect Detection using YOLO v8 model	46
B.5	Severity analysis of Detection	47

LIST OF ABBREVIATIONS

AI	:	Artificial Intelligence
CNN	:	Convolutional Neural Network
GSM	:	Global System for Mobile Communication
HIL	:	Hardware-in-the-Loop
LCD	:	Liquid Crystal Display
LED	:	Light Emitting Diode
ESP	:	Espressif (Microcontroller series, e.g., ESP32-CAM)
SoC	:	System on Chip
IDE	:	Integrated Development Environment
QVGA	:	Quarter Video Graphics Array
UAV	:	Unmanned Aerial Vehicle (Drone)
API	:	Application Programming Interface

CHAPTER 1
INTRODUCTION

1.1OVERVIEW

Bridges represent one of the most vital categories of civil infrastructure, facilitating transportation networks that underpin economic activity, emergency services, and daily commuting. Globally, hundreds of thousands of bridges are in active service, many of which were constructed decades ago and are now approaching or exceeding their originally designed service life. The structural integrity of these bridges is continuously compromised by a combination of factors including corrosive environmental exposure from moisture and chloride ingress, fatigue loading from vehicles and dynamic forces, freeze-thaw thermal cycling, and inadequate maintenance schedules.

Structural failures, even partial ones, can result in catastrophic consequences — loss of human life, destruction of property, disruption of supply chains, and substantial remediation costs. Notable historical incidents such as the Fern Hollow Bridge collapse in Pittsburgh (2022) and the Morandi Bridge collapse in Genoa (2018) underscore the critical importance of timely, accurate, and frequent structural inspection and intervention.

Traditional bridge inspection methodology relies on periodic visual assessments carried out manually by certified structural engineers. These inspections typically take place on a biannual or triannual cycle, meaning defects that develop and propagate between inspection cycles may go undetected until they become structurally critical. Furthermore, manual inspection of large bridges or elevated sections is logistically challenging, hazardous for inspectors, and prone to subjective variability based on the experience and condition of the assessor.

Recent advancements have introduced the use of Unmanned Aerial Vehicles (UAVs or drones) to capture imagery of difficult-to-access areas. However, post-capture image analysis in most current UAV-based systems remains a largely manual, offline activity, requiring trained engineers to review footage frame by frame. The deployment of high-performance GPU-based inference systems on-site to perform automated analysis is

computationally expensive and logistically impractical for widespread deployment across extensive national bridge networks.

The rapid proliferation of low-cost IoT hardware, combined with advances in deep learning-based computer vision, creates a compelling opportunity to build continuous, automated, edge-to-cloud bridge inspection pipelines. The ESP32-CAM is a compact, low-power, WiFi-enabled microcontroller module with an integrated OV2640 camera sensor, available for a fraction of the cost of professional inspection equipment.

This project proposes and implements such a system: BridgeGuard AI, an automated bridge defect detection and severity classification pipeline built around a fine-tuned YOLO v8 neural network, an ESP32-CAM edge node, a Flask web server backend, and a MongoDB-based data persistence layer. The system is capable of detecting five categories of structural surface defects — Corrosion, Crack, Free-lime, Leakage, and Spalling — and automatically assigning severity ratings based on a mathematically formulated spatial severity metric.

Additionally, the system is designed to support scalability across multiple bridge locations, enabling centralized monitoring dashboards and real-time alert generation for critical defects. This ensures proactive maintenance planning, reduces inspection delays, enhances safety compliance.

BridgeGuard AI demonstrates a practical and scalable approach to modernizing bridge inspection through the integration of IoT-enabled edge devices and deep learning-based defect analysis. By automating detection and severity assessment, the system minimizes human dependency, improves inspection frequency, and enables data-driven maintenance decisions. Its cost-effective architecture and adaptability make it suitable for large-scale deployment across diverse infrastructure networks. Ultimately, this approach contributes to enhanced structural safety, optimized resource allocation, and the long-term sustainability of critical transportation systems. The system incorporates a flexible architecture that can be extended to include additional sensing modalities such as vibration, temperature, and humidity sensors, enabling multi-modal structural health monitoring for more comprehensive and accurate diagnostics.

1.2 PROBLEM STATEMENT

Existing bridge inspection frameworks suffer from several compounding limitations that collectively reduce the reliability and coverage of structural health monitoring programs. Manual inspection is infrequent and subjective. With inspection cycles often spanning two to three years, rapidly progressing defects such as fatigue cracks and corrosion fronts may reach critical severity before being identified. Inspector fatigue, subjectivity, and limited access to certain structural zones further reduce detection reliability. Automated systems that exist are prohibitively expensive for widespread deployment. Vision-based systems that employ industrial cameras and on-site GPU inference nodes require significant capital investment, making it economically infeasible to instrument every bridge in a national network. Current edge-based IoT systems lack intelligence. Many IoT monitoring solutions collect raw sensor or image data and defer all analysis to a central server in batch mode, introducing significant delays between defect occurrence and alert generation. There is no standardized, automated mechanism for severity grading. Even when defects are detected, translating a raw detection into a maintenance priority score requires expert judgment, which reintroduces subjectivity and delays prioritization decisions.

The proposed system addresses all of these gaps by delivering a low-cost, continuously operating, automated defect detection and severity classification pipeline deployable on commodity hardware, ensuring real-time insights, scalability, efficiency, reliability, and improved infrastructure safety outcomes across diverse environmental conditions.

Furthermore, the proposed framework enhances decision-making by integrating real-time data analytics with a centralized monitoring interface, enabling infrastructure authorities to track defect progression over time and prioritize interventions based on quantified risk levels. The system’s modular design allows seamless integration with existing maintenance workflows and supports future upgrades, such as predictive maintenance using historical data trends. By reducing response time and enabling proactive intervention strategies, the solution significantly improves operational efficiency while ensuring consistent and objective assessment standards across large-scale bridge networks.

1.3 CATEGORY OF THE PROJECT:

1.3.1 Category I: Safety / Sustainability / Cost

The project falls under Category I as it ensures safety through early detection of structural defects such as cracks and corrosion, thereby reducing the risk of bridge failure. It improves inspection reliability by minimizing human error and enabling continuous monitoring. From a sustainability perspective, the system supports timely maintenance and extends the service life of infrastructure, reducing resource wastage. Additionally, the use of low-cost hardware components like ESP32-CAM and a centralized processing system makes the solution cost-effective, scalable, and suitable for large-scale deployment.

1.3.2 Category II: Application / Research

The project is categorized under Category II as it provides a practical application for real-time bridge defect detection using Artificial Intelligence and IoT technologies. It can be directly deployed for infrastructure monitoring in real-world scenarios. From a research perspective, the project involves the implementation of a YOLO v8-based deep learning model for multi-class defect detection and an automated severity assessment mechanism. The integration of computer vision and edge-to-cloud architecture contributes to advancements in intelligent structural health monitoring systems.

1.3.3 Relevance to Sustainable Development Goals (SDGs)

The proposed system aligns with **SDG 9: Industry, Innovation and Infrastructure** and **SDG 11: Sustainable Cities and Communities**. It enhances infrastructure reliability through the adoption of advanced technologies such as AI and IoT. By enabling early defect detection and proactive maintenance, the system improves public safety and reduces the risk of infrastructure failure. In addition, it supports data-driven decision-making by providing continuous monitoring data, enabling authorities to plan maintenance activities more efficiently**.** Overall, the project supports the development of safe, resilient, and sustainable urban infrastructure systems.

CHAPTER 2
LITERATURE SURVEY

2.1 INTRODUCTION

The intersection of computer vision, deep learning, and structural health monitoring has produced a growing body of research over the past decade. The literature reviewed in this chapter spans foundational defect detection methodologies, application of modern object detection architectures to civil infrastructure, edge computing approaches for IoT-based monitoring, and comparative evaluations of classical versus deep learning methods. The gap analysis derived from this review directly motivates the design decisions made in this project.

2.2 LITERATURE SURVEY

2.2.1 AI and Drone-based Bridge Defect Detection Using YOLO v8 Model

Deng et al. (2023) proposed a UAV-integrated visual inspection framework for highway bridges employing the YOLO v8 object detection model. High-resolution imagery was captured by drones flown along pre-programmed flight paths at varying altitudes and angles to ensure comprehensive surface coverage. The YOLO v8 model was trained on a custom dataset and demonstrated strong detection recall for crack and spalling defects across steel and concrete bridge surfaces. The system achieved real-time inference at 30 frames per second on onboard edge hardware. However, the authors noted that detection accuracy degraded significantly under adverse weather conditions and for subsurface defects not visible to optical sensors. The study did not incorporate a quantitative severity grading mechanism, limiting its utility for maintenance prioritization.

2.2.2 Defect Classification Using Deep Learning

Zhang et al. (2022) conducted a comprehensive comparative study of multiple deep learning architectures — including ResNet-50, VGG-16, EfficientNet-B4, and MobileNetV2 — applied to the binary and multi-class classification of bridge surface defects from standard photographic imagery. Their dataset comprised approximately 8,000 annotated images collected from bridge inspection archives. The study found that EfficientNet-B4 achieved the highest classification accuracy (94.2%) but required the

longest inference time, whereas MobileNetV2 offered a favorable trade-off between accuracy (89.7%) and computational efficiency, making it suitable for resource-constrained deployments. The paper did not address real-time streaming architectures or IoT integration.

2.2.3 Implementation of AI and Drone-based Bridge Defect Detection

Liu et al. (2023) implemented an end-to-end inspection pipeline combining drone-captured imagery with a server-side YOLO-based detection model deployed through a REST API. Their architecture enabled inspectors to upload drone footage segments for automated defect localization, receiving annotated output frames with bounding boxes and class labels within seconds. The implementation demonstrated that cloud-offloaded deep learning inference could effectively support field inspection workflows. A notable limitation was the reliance on manual drone operation and post-flight upload, rather than continuous automated streaming, and the absence of database-backed historical defect tracking.

2.2.4 Object Classifier with Image Processing and YOLO v8 Model

Chauhan et al. (2023) explored the application of YOLO v8 as a general-purpose object classifier for industrial quality control applications, augmenting raw model outputs with image preprocessing steps including histogram equalization, Gaussian denoising, and adaptive thresholding. Their experiments showed that preprocessing significantly improved detection recall for low-contrast defect appearances (e.g., hairline cracks on light-colored concrete). The study validated the effectiveness of YOLO v8 for multi-class classification tasks with small inter-class visual differences, a challenge directly relevant to the five-class defect taxonomy used in this project.

2.2.5 AI-Powered Bridge Defect Classifier and Severity Analysis

Wang et al. (2023) proposed a bridge defect classification system augmented with a rule-based severity scoring module. Their system used a custom CNN classifier to identify defect types and then applied dimensional analysis on bounding box geometry to assign severity scores on a three-tier scale.

Table 2.1 : Comparison on Waste Classification Models

Title of the Paper & Year of Publication	Proposed Algorithm	Parameters	Tools Used for Implementation	Limitations
Improved YOLO v8n-based bridge crack detection algorithm under complex background conditions (2025).	Global Attention and Gam-Concat fusion, MPDIoU Loss Function, CNN-based Crack Detection,Data Augmentation & Preprocessing.	YOLO v8n Gam-Concat, MPDIoU loss	YOLO v8n Pytorch OpenCV GPU acceleration(CUDA	Still struggles with very thin or dense cracks under complex noisy background
Concrete Surface Crack Detection Algorithm Based on Improved YOLO v8 (YOLO v8-CD)-2024	Visual Attention Networks Ghost Module GSConv ,VoV-GSCSP	**Datasets:** RDD2022, WallCrack **Speed:** 88 FPS	YOLO v8 PyTorch OpenCV GPU	Model trained on specific datasets may not adapt to all real-world structures with varying materials, lighting,or noise.

LCA YOLO v8-Seg:An Improved Lightweight YOLO v8-Seg for Real-Time Pixel-Level Crack Detection of Dams and Bridges-2023	Lightweight Backbone LCA (Large Convolutional Attention) YOLO v8-Seg	Accuracy: 0.945 Speed: 129 FPS Efficiency: Smaller model size	YOLO v8-seg OpenCV Pytorch GPU	Noisy background due to thin cracks. Generalization limitation
Advanced bridge visual inspection using real-time machine learning in edge devices-2022.	Faster R-CNN YOLO F1-score	Lightweight architecture	PyTorch UAVs(Edge device)RDD2022(Dataset)	Generalization &Dataset Bias
Automated bridge crack detection method based on lightweight vision models-2022	Faster R-CNN Segement PSPNet(DLF)+ CNNs	Accuracy – 98% PSPNet Accuracy = 97.86%	TensorFlow, Keras 2068 bridge crack images (split 8:1:1)	Framework Dependency Dataset Size & Diversity

Research on Efficiency of Bridge Crack Detection by Coupling Deep Learning Frameworks-2023.	Faster R-CNN **Segmentation Models:**PSPNet.(DLF)+CNNs	Accuracy:98% .PSPNet Accuracy = 97.86%.	TensorFlow2, Keras 2068 bridge crack images (split 8:1:1)	Framework Dependency Dataset Size & Diversity
Real-Time Detection of Cracks on Concrete Bridge Decks using 1D-CNN + LSTM (freq domain)-2021.	1D-CNN LSTM Frequency Domain Preprocessing	Accuracy 93 % computation time compared to existing methods.	Pytorch Transformation of crack images.GPU	Generalization Issue Scalability Challenge .

Table 2.1 presents a comparative analysis of various bridge crack and defect detection models based on deep learning techniques. The comparison includes different algorithms, performance parameters, implementation tools, and their respective limitations.

CHAPTER 3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The conventional approach to bridge structural inspection relies on a combination of scheduled manual visual surveys, physical contact-based non-destructive testing (NDT), and periodic UAV-assisted imagery collection. In the manual inspection model, certified structural engineers visit bridge sites at intervals prescribed by regulatory standards, typically every 24 to 36 months. Inspectors visually examine exposed structural surfaces, recording observations through handwritten field notes, photographs, and standardized condition rating forms.

Several significant drawbacks are inherent to this model. The periodicity of inspection means that defects occurring between cycles are invisible to the monitoring system until the next scheduled visit. Manual observation is subjective — different inspectors may rate the same crack differently based on individual experience, fatigue, and assessment methodology. Accessing elevated bridge components such as under-deck beams, cable anchor zones, and pier faces requires expensive scaffolding, rope access, or bucket truck deployment, elevating both cost and personnel risk. The subsequent digitization and analysis of inspection records is also labor-intensive and creates delays between field observation and maintenance decision-making.

More recent systems have introduced UAV-based imagery capture, which reduces personnel risk for accessing difficult areas. However, the image analysis step in most current UAV-based systems remains a largely manual, offline activity. The collected footage or photographs are reviewed by engineers after the inspection flight, requiring the same level of expert human judgment as a conventional visual survey. On-site automated analysis using deep learning is technically feasible but requires deploying GPU-equipped edge servers or industrial-grade embedded hardware at each inspection site, which is prohibitively expensive at scale.

Additionally, existing systems lack a standardized, automated severity quantification mechanism. Even when defects are visually identified, translating a raw observation into a

maintenance priority score requires further expert deliberation, introducing additional delay and inconsistency into the maintenance workflow.

Disadvantages of Existing System

- Infrequent, periodic inspections leave gaps in structural health coverage
- Human subjectivity introduces inconsistency in defect classification and severity assessment
- High cost and safety risk associated with manual access to structural components
- Post-flight manual analysis of UAV imagery lacks automation
- No standardized automated severity scoring or database-backed defect history tracking
- Real-time alerting for emerging critical defects is not supported

3.2 PROPOSED SYSTEM

BridgeGuard AI is proposed as a low-cost, continuous, automated bridge defect detection and severity quantification system built on an edge-to-cloud architecture. The system employs an ESP32-CAM IoT module as a permanently deployed or periodically positioned visual capture node. The camera captures JPEG-compressed images at regular 1-second intervals and transmits them over WiFi via HTTP POST requests to a centralized Flask web server.

On the server, a fine-tuned YOLO v8 model performs multi-class defect detection, identifying instances of five defect categories — Corrosion, Crack, Free-lime, Leakage, and Spalling — within each frame. For each detection, a severity score is automatically computed using the proposed Relative Area Severity metric, which combines bounding box spatial coverage ratio and model confidence score to assign one of four severity tiers: Low, Medium, High, or Critical. Detection results, annotated images, severity grades, and geospatial coordinates are persisted in a MongoDB database. A web-based dashboard provides real-time visualization of detection activity, historical trend analysis, and PDF report generation.

Advantages of Proposed System

- Continuous, near real-time monitoring without manual inspection cycles

- Fully automated defect classification using a trained deep learning model, eliminating subjectivity
- ESP32-CAM hardware cost is a fraction of professional inspection equipment
- Automated, mathematically formulated severity scoring enables objective maintenance prioritization
- All inspection records are persistently stored with timestamps for longitudinal structural health analysis
- The system is scalable — multiple ESP32-CAM nodes can report to the same server
- PDF report generation enables direct integration with maintenance management workflows.

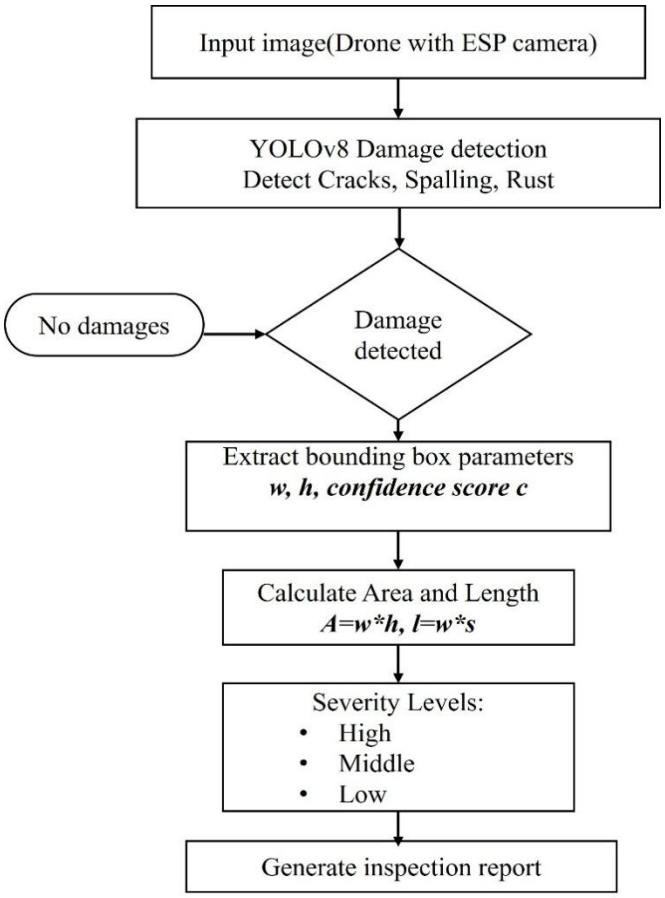


Figure 3.1 : Workflow Diagram of Defect classification

CHAPTER 4

SYSTEM SPECIFICATION

4.1 HARDWARE CONFIGURATION

The hardware used for the development of the project is:

PROCESSOR(SERVER-SIDE)	:	CPU BASED SYSTEM(LAPTOP)
RAM	:	4 GB(MINIMUM)
MONITOR	:	HDMI-COMPATIBLE MONITOR
STORAGE	:	32 GB +
CAMERA MODULE	:	ESP CAM 32 MODULE(OV2640 CAMERA SENSOR)

4.2 SOFTWARE CONFIGURATION

The software used for the development of the project is:

OPERATING SYSTEM	:	WINDOWS/LINUX
ENVIRONMENT	:	VISUAL STUDIO CODE
LANGUAGE	:	PYTHON 3.x
LIBRARIES USED	:	FLASK.ULTRALYTICS YOLO v8, OPENCV,TENSORFLOW
DATABASE	:	SQLITE

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 PYTHON 3.X

Python is a high-level, interpreted, general-purpose programming language known for its clean and readable syntax, extensive standard library, and a vast ecosystem of third-party packages for scientific computing, machine learning, and web development. In this project, Python 3.10 serves as the primary development language for all server-side components, including the Flask API server, YOLO v8 inference pipeline, image processing routines, database integration logic, and PDF report generation modules. Python's compatibility with the Ultralytics YOLO v8 library, PyTorch, OpenCV, and PyMongo makes it the natural and unified language choice for the entire backend stack.

5.2 TENSORFLOW & KERAS

TensorFlow is an open-source deep learning framework developed by Google, and Keras is a high-level API that runs on top of TensorFlow. Together, they enable the development, training, and deployment of the YOLO V8 model used in this project. YOLO v8 is chosen because it is efficient and lightweight, making it perfect for edge devices like Raspberry Pi. The model is trained on images of recyclable, non-recyclable, and hazardous waste, allowing it to classify them accurately in real time.

TensorFlow and Keras simplify the creation of convolutional neural networks (CNNs) with just a few lines of code. They offer utilities like callbacks, image augmentation, and fine-tuning to boost model accuracy. The trained model can be saved and reused for real-time inference with minimal latency. These tools also support GPU acceleration during training if available, speeding up model development.

5.3 OPENCV

OpenCV (Open Source Computer Vision Library) is used in the project to interact with the camera module and handle image processing. It captures frames in real-time and processes them by resizing, converting to arrays, and normalizing before feeding them into the machine learning model. OpenCV is lightweight and efficient, making it ideal for embedded systems like ESP cam32. It supports various camera types—USB webcams or Pi

cameras—ensuring flexibility in hardware choice.

It also enables features like live video feed, image cropping, and grayscale conversion. The image captured by OpenCV is stored temporarily for classification and then optionally uploaded to MongoDB. Its integration with NumPy allows for high-performance matrix operations essential in computer vision. In this project, OpenCV acts as the bridge between hardware (camera) and AI (model). It's reliable for continuous monitoring and ensures images are correctly prepared for the next stage of classification.

5.4 YOLO V8 MODEL

YOLO v8 (You Only Look Once, Version 8), developed by Ultralytics, represents the current state of the art in single-stage real-time object detection. The architecture employs an anchor-free detection paradigm with a decoupled detection head and a CSP-based (Cross Stage Partial) C2f backbone, enabling superior feature extraction with reduced parameter count. YOLO v8 is available in multiple size variants (nano, small, medium, large, extra-large) offering scalable precision-speed trade-offs. In this project, the YOLO v8s (small) variant was selected for fine-tuning on the bridge defect dataset, providing a balance between detection accuracy and inference speed acceptable on CPU-based server hardware. The model is loaded using the Ultralytics Python library and called directly through the model.predict() API on each received frame.

5.5 SQLite

SQLite is a lightweight, embedded relational database management system that stores data in a structured tabular format within a single local file. Unlike NoSQL databases, SQLite follows a predefined schema, ensuring consistency and integrity of stored inspection records while maintaining minimal system overhead.In the proposed BridgeGuard AI system, SQLite is used to store all inspection-related data, including frame timestamps, detected defect class labels, bounding box coordinates, confidence scores, computed severity levels, estimated lifespan impact, and approximated geospatial coordinates. Each inspection event is recorded as structured entries across relational tables, enabling efficient querying and retrieval.

SQLite is particularly suitable for this project due to its serverless architecture, which

allows seamless integration with the Flask-based processing server without requiring additional database configuration. This ensures fast write operations and low latency during real-time inference workflows.Its structured design supports efficient indexing and querying, enabling functionalities such as historical inspection tracking, temporal trend analysis, and automated generation of PDF-based inspection reports. Despite being lightweight, SQLite can handle moderate volumes of structured data efficiently, making it ideal for prototype and mid-scale deployments of infrastructure monitoring systems.Furthermore, SQLite integrates smoothly with Python using built-in libraries such as `sqlite3`, ensuring reliable data communication and simplified database management within the application. For large-scale deployments, the system can be extended to more robust database solutions; however, SQLite serves as an optimal choice for the current implementation due to its simplicity, portability, and performance efficiency.

5.6 VISUAL STUDIO CODE

Visual Studio Code is used as development environments for writing and debugging Python scripts. VS Code offers rich extensions, linting, and integration with Git, SSH, and Jupyter Notebooks. It can be used to develop remotely on the Raspberry Pi via SSH, allowing for real-time edits and execution. This IDE provide a platform to run training scripts, test model inference, and check database connectivity. VS Code help track down bugs using breakpoints and console outputs. The use of this IDE ensures efficient software development and testing throughout the project lifecycle. This IDE supports execution of training scripts, testing of YOLO v8 model inference, and verification of database connectivity with MongoDB. It also provides powerful debugging tools such as breakpoints, variable inspection, and console logging, which help in identifying and resolving errors efficiently.
In addition, VS Code supports environment management using virtual environments, ensuring proper dependency handling for libraries like OpenCV, PyTorch, and Ultralytics. It also allows API testing through integrated terminals and extensions like Postman or REST Client.The use of VS Code ensures a structured and efficient development workflow throughout the project lifecycle, improving productivity, code maintainability, and collaborative development.

5.7 ESP CAM 32

The ESP32-CAM is a compact microcontroller module based on the Espressif ESP32 SoC (System on Chip), featuring an integrated OV2640 camera sensor and an IEEE 802.11 b/g/n WiFi radio. Despite its small form factor, the ESP32 dual-core processor running at 240 MHz is capable of capturing JPEG-compressed images, managing WiFi connectivity, and executing HTTP POST requests to upload images to a remote server. In this project, the ESP32-CAM module was programmed using the Arduino framework to capture images at QVGA resolution (320×240) with JPEG quality level 14, producing payloads typically under 30KB to minimize transmission latency over constrained wireless networks. The module operates on a 1 Hz timer interrupt, continuously uploading frames to the Flask server for real-time analysis. Its sub-$10 unit cost makes it an economically viable basis for deploying distributed monitoring nodes across bridge inspection sites.LED or motor control based on waste classification. Raspberry Pi OS includes built-in Python and Thonny IDE, making it ready out-of-the-box for IoT development.

It is optimized to manage system resources efficiently, even with AI workloads. Regular updates and community support ensure security and compatibility. The OS supports internet connectivity via Wi-Fi or Ethernet, essential for syncing with cloud databases like MongoDB Atlas. Its GUI is user- friendly for basic system operations, while terminal access allows for advanced scripting and automation. It forms the backbone for deploying the smart trash can as a self-contained intelligent edge device.

Furthermore, the ESP32-CAM module plays a crucial role in enabling real-time data acquisition in the proposed system. Its ability to operate as a low-power, wireless image capture device makes it highly suitable for deployment in remote or hard-to-access bridge locations. The seamless integration with the Flask-based server ensures efficient data transmission and processing, while maintaining minimal latency. Due to its compact size, affordability, and reliable performance, the ESP32-CAM serves as an effective edge device for scalable and continuous infrastructure monitoring application

CHAPTER 6

PROJECT DESCRIPTION

6.1 PROBLEM IDENTIFICATION

The core problem addressed by this project is the absence of a low-cost, continuous, automated pipeline for bridge structural defect detection and severity quantification that is deployable using commodity IoT hardware. Existing solutions either rely on expensive on-site hardware for real-time inference or defer analysis to manual post-inspection review. The specific technical challenges solved include: real-time image streaming from a constrained edge device over a wireless network, multi-class structural defect detection using a deep learning model tuned for the inspection domain, automated severity quantification without stereo depth calibration, and persistent storage and reporting of defect records for longitudinal analysis.

6.2 LIST OF MODULES

1. Image Acquisition
2. Image Processing
3. Defect Classification
4. Severity Analysis
5. Database Management
6. User Interface & Output

6.3 MODULE DESCRIPTION

6.3.1 Image Acquisition

The Image Acquisition module is implemented in firmware running on the ESP32-CAM microcontroller, written using the Arduino framework in C/C++. Upon power-on, the module initializes the OV2640 camera sensor with the following parameters: frame size QVGA (320×240), JPEG quality level 14 (high compression), and a 1 Hz capture-and-transmit loop driven by a hardware timer interrupt. At each trigger, the module captures a JPEG frame, constructs an HTTP POST request with multipart/form-data encoding containing the image payload, and transmits it to the configured Flask server endpoint over the WiFi network. The module is pre-configured with the target server IP address and port,

and implements basic error handling to retry failed transmissions. LED indicators on the ESP32-CAM module signal successful capture and transmission events.

6.3.2 Image Preprocessing

The Image Processing module on the server side receives the raw JPEG payload from the HTTP POST request, decodes it into a NumPy array using OpenCV's cv2.imdecode() function, and applies basic preprocessing operations before inference. These operations include verifying image integrity, recording image dimensions (width and height) for subsequent severity ratio computation, and generating a unique timestamp-based filename for storage. After YOLO v8 inference, the module renders bounding box overlays, class label text, and severity tier indicators onto the annotated output frame using OpenCV drawing primitives, producing a visualization suitable for dashboard display. The annotated frame is saved to a designated output directory under the prefix.

6.3.3 Defect Classification

The Defect Classification module loads the fine-tuned YOLO v8s model weights at server startup and exposes a prediction function invoked on each received frame. The model is called via `model.predict(image, conf=0.10, iou=0.45)`, where the low confidence threshold of 0.10 maximizes recall on initial detection passes, ensuring that potential defects are not missed at the cost of some false positives. For each bounding box in the prediction output, the module extracts: the bounding box coordinates (x1, y1, x2, y2), the associated class label (one of: Corrosion, Crack, Free-lime, Leakage, Spalling), and the model confidence score. These values are passed to the Severity Analysis module for further processing.

6.3.4 Database Management

The Database Management module handles all interactions with the SQLite database. Upon server startup, a connection is established to a local SQLite database file using a Python interface such as sqlite3. After each inference cycle, the module prepares a structured record containing the frame timestamp, image filename, and serialized detection data (including class, confidence, severity, lifespan impact, and geospatial coordinates). This data is inserted into the corresponding tables (such as inspections and detections) using SQL

INSERT statements.

The module also provides query functions to retrieve recent inspection records for dashboard visualization, along with aggregation queries to support report generation and historical trend analysis. By leveraging SQLite, the system remains lightweight and efficient, making it suitable for edge or low-resource environments. Additionally, the database can be synchronized with a cloud server when required, enabling remote access, alert triggering, and advanced analytics. This module supports future enhancements such as automated reporting, trend prediction, and real-time visualization.

6.3.5 User Interface & Output

The User Interface module is implemented as a set of Flask HTML template routes served to a web browser. The dashboard presents a real-time feed of annotated inspection images, a tabular view of recent detections with class, confidence, severity, and location fields, and historical severity distribution charts. A map view displays geospatially tagged detection markers, providing a spatial overview of defect distribution across the monitored bridge section. A report generation button triggers PDF compilation via the report generation module, producing a structured inspection summary document downloadable from the dashboard. The interface is styled for field-use readability, with clear severity color coding: green for Low, yellow for Medium, orange for High, and red for Critical.

6.3.6 Pseudocode

Algorithm 1: Server-Side Defect Analysis Pipeline

Input: ImageFile (Multipart HTTP POST from ESP32-CAM)

Output: Annotated Image, MongoDB Record, JSON Response

FUNCTION PROCESS_INSPECTION_PAYLOAD(ImageFile):

filename ← GenerateTimestampName(ImageFile)

SaveFile(ImageFile, filename)

Image ← ReadImage(filename)

Area_img ← Image.height × Image.width

predictions ← YOLO v8.predict(Image, conf=0.10, iou=0.45)

DetectionsList ← []

```
FOR each BBox IN predictions:
DetectionsList ← []
  x1, y1, x2, y2 ← BBox.coordinates
  Conf ← BBox.confidence
  Label ← BBox.class_name
  Area_box ← (x2 - x1) × (y2 - y1)
  Ratio ← Area_box / Area_img
  IF Ratio > 0.10 AND Conf > 0.80:
    Severity ← "Critical"
  ELSE IF Ratio > 0.10 AND Conf ≤ 0.80:
    Severity ← "High"
  ELSE IF 0.05 < Ratio ≤ 0.10:
    Severity ← "Medium"
  ELSE:
    Severity ← "Low"
  LifespanImpact ← EstimateLifespan(Severity)
  Lat, Lon ← ApplySpatialJitter(Base_Lat, Base_Lon)
  RenderOverlay(Image, BBox, Label, Severity)
  DetectionsList.Append({Label, Conf, Severity, LifespanImpact, Lat, Lon})
END FOR
IF DetectionsList IS EMPTY:
LogEvent("No defects detected", filename)
RETURN {"status": "OK", "message": "No defects found"}
AnnotatedPath ← SaveAnnotatedImage(Image, filename)
Record ← {
"filename": filename,
"detections": DetectionsList,
"timestamp": CurrentTime(),
```

"image_path": AnnotatedPath

}

MongoDB.Insert("inspection_collection", Record)

JSONResponse ← Serialize({

"status": "success",

"detections": DetectionsList,

"image_url": AnnotatedPath

})

RETURN JSONResponse

END FUNCTION

This pseudocode describes the automated operation of a Smart Trash Can using Raspberry Pi 4 and a deep learning model. When waste is detected, it captures an image, classifies it using the MobileNetV2 model, and activates the appropriate servo motor to sort the waste into recyclable, non-recyclable, or hazardous bins. Finally, it logs the classification result along with the timestamp and image into MongoDB for future analysis. Based on the classification result, the corresponding servo motor activates to open the appropriate waste bin, allowing the item to be segregated correctly. Each classification result, along with a timestamp and the image data.

This pseudocode describes the automated operation of a server-side defect analysis pipeline in the proposed BridgeGuard AI system. When an image is received from the ESP32-CAM module, it is processed using the YOLO v8 model to detect structural defects. Based on the detected bounding boxes, the system calculates the severity level using area ratio and confidence score. The annotated image is generated, and all detection results along with timestamps and geospatial data are stored in MongoDB for future analysis and monitoring.

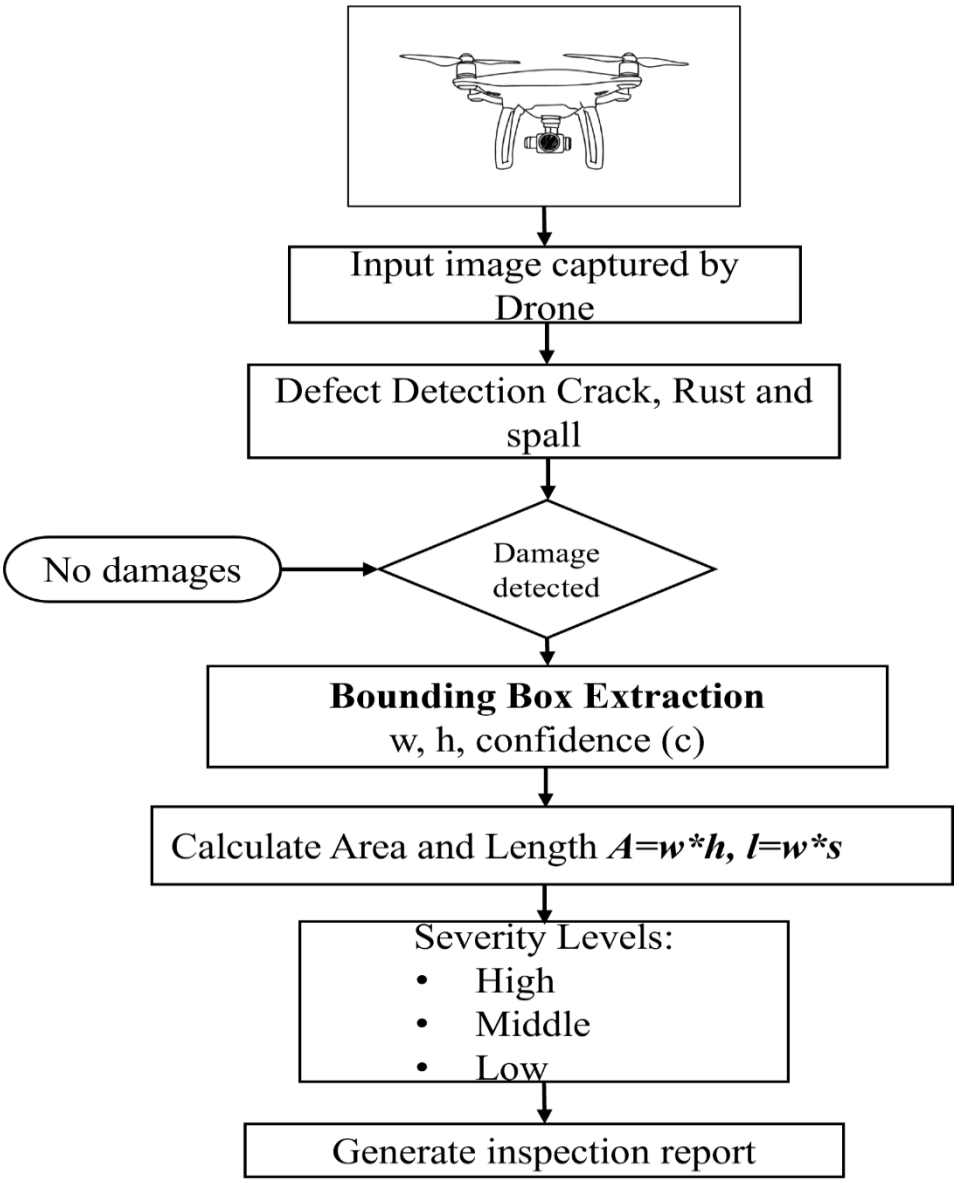


Figure 6.1 : Workflow Diagram of Defect classification

The figure 6.1 illustrates the overall workflow of the proposed bridge defect detection system. Initially, images of the bridge surface are captured using a drone, which enables access to difficult and elevated areas. These input images are then processed by the defect detection module, where various types of structural damages such as cracks, rust (corrosion), and spalling are identified.

A decision stage follows, where the system determines whether any damage is present. If no defects are detected, the process ends with a "No damages" output. If defects are identified, the system proceeds to extract bounding box parameters, including width (w), height (h), and confidence score (c), which represent the detected defect region.

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 IMPLEMENTATION

The BridgeGuard AI system was implemented in two parallel tracks: embedded firmware development for the ESP32-CAM edge node, and Python-based backend development for the Flask inference server. Both tracks were integrated and tested end-to-end in a controlled lab environment before field evaluation. The implementation followed an iterative development methodology, with each module incrementally tested before integration into the full pipeline.

7.2 EXPERIMENTAL SETUP

The experimental setup simulates a real-world bridge inspection deployment. The ESP32-CAM module was positioned at varying distances (0.5m to 2m) from printed and physical concrete surface samples exhibiting different defect types, replicating the standoff distances achievable in a close-range bridge inspection scenario. The WiFi network used for transmission was a standard 2.4 GHz 802.11n access point. The Flask inference server ran on a laptop equipped with an Intel Core i7 processor and 16GB RAM, performing CPU-based YOLO v8 inference. Average end-to-end latency from image capture to JSON response receipt at the ESP32-CAM was measured at approximately 1.2 to 1.8 seconds under typical network conditions, confirming near real-time performance.

7.3 SOFTWARE DEVELOPMENT

The software development of the Smart Trash Can involves several key components, including embedded system programming, machine learning model integration, real-time classification, and database management. The software is developed using Python and deployed on a ESP cam 32, which serves as the central processing unit. The entire software architecture is designed to handle real- time waste classification, sensor control, and data logging

7.3.1 Embedded System Programming

The ESP32-CAM firmware was developed in C/C++ using the Arduino IDE with the Espressif ESP32 board support package. The firmware implements: WiFi initialization and connection management using the WiFi.h library, camera sensor initialization using the esp_camera.h ESP-IDF camera driver, a timer-based 1 Hz capture loop using esp_timer_create(), and HTTP POST transmission using the HTTPClient.h Arduino library. The JPEG image buffer captured by the camera sensor is directly passed as the HTTP POST body without additional encoding, minimizing processing overhead on the microcontroller. Error handling includes automatic reconnection on WiFi dropout and frame discard on capture failure.

7.3.2 Machine Learning Model Integration

The YOLO v8 model integration on the server side uses the Ultralytics Python library. After training (described in Section 7.4), the best-performing model weights file (best.pt) is loaded at application startup via YOLO('best.pt'). For each incoming image, the models predict () method is invoked with a confidence threshold of 0.10 and IoU threshold of 0.45. The resulting Results object contains a list of Boxes objects, each providing the bounding box tensor, class index, and confidence score. These values are extracted and post-processed through the severity computation logic before database insertion.

7.3.3 Backend and API Development

The Flask application defines two primary routes: a POST endpoint at /upload that receives the ESP32-CAM image payload, invokes the inference pipeline, and returns a JSON response; and a GET endpoint at /dashboard that renders the web interface template populated with recent inspection records retrieved from MongoDB. Additional routes support PDF report download (/report), annotated image serving (/images/<filename>), and a JSON API for historical data retrieval used by the dashboard's visualization components. The application is run in debug mode during development and configured for production deployment using a WSGI server.

7.3.4 Data Management

The MongoDB database is used to store the waste classification records for future analysis and monitoring. Each entry includes a timestamp indicating when the waste was disposed, the predicted category such as recyclable, non-recyclable, or hazardous, and the confidence score from the AI model. This data helps evaluate model performance over time and supports further improvements. MongoDB's flexibility and scalability make it ideal for handling structured classification data efficiently. MongoDB’s document-based architecture ensures fast data insertion and retrieval, which is crucial for real-time applications. Its flexibility allows seamless adjustments to the data schema as the system evolves. Over time, the collected data can be analyzed to identify trends and optimize waste management strategies. The database also supports remote access, enabling centralized monitoring of multiple smart bins.

Table 7.1 : The Image Processing of the YOLO v8 Model

Input	Processing	Output
Image captured by ESP32-CAM	Image is compressed (JPEG), resized (QVGA), and transmitted via HTTP POST	Image received at Flask server for processing
Incoming image at server	Image is decoded and optionally resized to 640×640 for model compatibility	Preprocessed image ready for inference
Preprocessed image	Ultralytics YOLO v8 extracts features and performs object detection	Detected defect classes (Crack, Spalling, Corrosion, Leakage, Free-lime) with

		bounding boxes
Detection results	Bounding box coordinates and confidence scores are computed	Structured detection data generated
Detection data	Severity calculated using area ratio and confidence logic	Severity levels (Low, Medium, High, Critical) assigned
Processed detections	Geospatial mapping applied (base GPS + jitter approximation)	Approximate defect location (Latitude, Longitude)
Annotated results	Bounding boxes and labels rendered using OpenCV	Annotated output image generated

7.4 MACHINE LEARNING MODEL TRAINING

The YOLO v8 model is used for waste classification due to its lightweight architecture, efficiency on edge devices, and high accuracy. The model is trained using Transfer Learning, where a pre-trained YOLO v8 is fine-tuned with a custom dataset of labeled waste images. The training and testing process involves four major steps:

7.4.1 Data Collection

The data collection process involves gathering a diverse set of waste images from various sources to ensure accurate classification. This includes manually capturing images using a USB camera under different lighting and background conditions to simulate real-world scenarios. Additionally, publicly available datasets related to waste classification are incorporated to enrich the dataset. To further enhance model generalization, image augmentation techniques such as rotation, flipping, and scaling are applied. These variations help the AI model recognize waste types more accurately across different environments and orientations.

7.4.2 Data Preprocessing

All images were preprocessed using Roboflow's preprocessing pipeline prior to download. Preprocessing steps included: EXIF auto-orientation to correct camera rotation metadata, uniform resize to 640×640 pixels using stretch-to-fit interpolation to match YOLO v8's expected input dimensionality, and dataset splitting into Training (70%, 2193 images), Validation (20%, 628 images), and Test (10%, 313 images) subsets with stratified class distribution. Data augmentation applied during training included horizontal flipping, mosaic composition (combining four images), random HSV color jitter, and small-degree rotation to improve model generalization across varying inspection angles and lighting conditions.

7.4.3 Model Training (Transfer Learning)

Training was conducted using transfer learning from YOLO v8s ImageNet-pretrained weights, fine-tuned on the bridge defect dataset for 100 epochs with a batch size of 16. The AdamW optimizer was used with an initial learning rate of 0.01 and cosine learning rate decay. Training was performed on a system with an NVIDIA GPU (CUDA-enabled), completing in approximately 3.5 hours. The training script used the Ultralytics CLI: yolo detect train model=YOLO v8s.pt data=bridge.yaml epochs=100 imgsz=640.

7.4.3 Model Selection (YOLO v8):

YOLO v8, a lightweight and efficient convolutional neural network developed by Google, was selected for this project. It was chosen for its high performance on mobile and embedded systems like the ESP cam 32. It uses depthwise separable convolutions, which significantly reduce computation while maintaining accuracy. The model was initialized with pre- trained weights from ImageNet to leverage transfer learning, allowing it to adapt quickly to the waste classification task with minimal training time. This approach ensured the model remained lightweight yet powerful for real-time image recognition.

7.4.4 Model Evaluation:

Once training was complete, the model was evaluated using the test dataset to measure its real-world performance. Metrics such as accuracy, precision, recall, and F1-score were calculated to assess the classification quality. Confusion matrices were generated to

visualize how well the model distinguished between different waste categories. The evaluation confirmed that the model achieved satisfactory accuracy levels, making it suitable for deployment. Misclassified images were reviewed to understand potential weaknesses and improve future versions of the model. The model was integrated with the camera and servo motor control logic so that each classification result triggered the appropriate bin opening. The seamless deployment ensured that the model could operate in real time, making waste disposal more efficient and intelligent.

7.5 SYSTEM INTEGRATION

Full system integration was performed by co-deploying the ESP32-CAM firmware and the Flask server, connecting both to the same WiFi network, and executing an end-to-end test with the camera positioned above physical concrete surface samples. Each captured frame was transmitted, analyzed, annotated, stored in MongoDB, and visible on the live dashboard within approximately 1.5 seconds of capture. Integration testing confirmed correct operation of all modules: image reception, inference, severity calculation, database insertion, annotated image storage, and dashboard rendering.

Table 7.2: Waste Identification of YOLO v8 Model

Input	Processing	Output
Bridge surface image captured by ESP32-CAM	Image captured and compressed (JPEG), transmitted via HTTP	Image received at server for analysis
Captured image	Image preprocessing (decoding, resizing to 640×640, normalization)	Image prepared for detection
Preprocessed image sent to AI model	Ultralytics YOLO v8 extracts features and detects defects	Predicted defect categories (Crack, Spalling, Corrosion, Leakage, Free-lime)

Detection results	Bounding boxes and confidence scores computed	Structured detection outputs generated
Detection output	Severity calculated using area ratio + confidence logic	Severity levels (Low, Medium, High, Critical)
Processed results	Annotated using OpenCV (bounding boxes + labels)	Visual output with highlighted defects

Table 7.2 presents the step-by-step workflow of the YOLO v8-based bridge defect detection process, starting from image acquisition to final output generation. It clearly illustrates how raw images captured by the ESP32-CAM are transmitted, preprocessed, and analyzed using deep learning techniques.

The table highlights the transformation of input data through multiple stages such as image decoding, resizing, and normalization to ensure compatibility with the YOLO v8 model. The model then performs feature extraction and accurately identifies multiple defect categories present on bridge surfaces.

Furthermore, the YOLO v8 model contributes to high detection accuracy due to its advanced deep learning architecture, which balances speed and precision. The model is capable of detecting multiple defects simultaneously within a single frame, improving overall inspection efficiency compared to traditional manual methods. The integration of confidence scoring also helps in filtering uncertain predictions, thereby increasing the reliability of the output results.

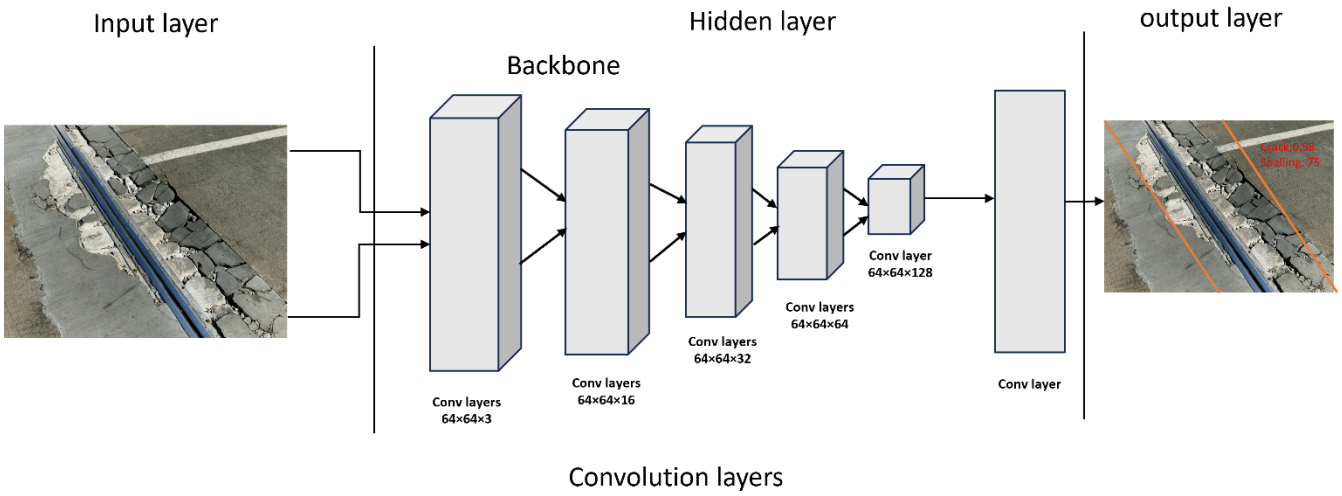


Figure 7.1 : Workflow Diagram of YOLO v8 Model

Figure 7.1 portrays the complete workflow of an AI-based bridge defect detection system. On the left, a raw image captured from the bridge surface serves as the input. This image is then passed through multiple stages of a deep learning model, represented by the series of blocks in the center, where features such as edges, textures, and structural patterns are progressively extracted and refined. As the data moves through these layers, the model identifies and classifies potential defects. The tall block near the right indicates the final detection stage, where bounding boxes, labels, and confidence scores are generated. Finally, the output image on the right shows the annotated result, where defects like cracks and spalling are clearly marked along with their confidence values, demonstrating how the system transforms an unprocessed image into meaningful inspection insights. The tall block near the right indicates the final detection stage, where bounding boxes, labels, and confidence scores are generated. Finally, the output image on the right shows the annotated result, where defects like cracks and spalling are clearly marked along with their confidence values, demonstrating how the system transforms an unprocessed image into meaningful inspection insights. **Overall, this pipeline enables automated, accurate, and efficient bridge inspection, reducing manual effort and improving infrastructure safety.**

CHAPTER 8

SYSTEM TESTING

8.1 INTRODUCTION:

System testing was conducted to validate the correctness, reliability, and performance of all system components individually and in integration. The testing strategy combined unit testing of individual server-side functions, hardware-in-the-loop testing of the ESP32-CAM to server communication pipeline, functional test cases for end-to-end defect classification and database storage, and comparative performance analysis against manual and baseline automated inspection methods.

8.2UNIT TESTING:

Unit testing is a fundamental software testing method used to validate individual components or modules of the Smart Trash Can system. In this project, unit tests are applied to separate functions such as image preprocessing, model prediction, sensor input reading, and database insertion logic. Each function is tested in isolation to ensure it performs as expected, without interference from other parts of the system. For example, the waste classification function is tested using dummy image inputs to verify correct label outputs. Similarly, the MongoDB storage module is tested to confirm that data such as timestamp, confidence score, and classification are properly inserted. This method increases code reliability and simplifies debugging, as errors can be traced directly to specific modules.

8.3HARDWARE-IN-THE-LOOP TESTING (HIL):

Hardware-in-the-loop testing was performed by operating the actual ESP32-CAM module, transmitting live frames to the Flask server over a real WiFi network. Test scenarios included: normal operation at 1 Hz transmission rate, behavior under intermittent WiFi connectivity (disconnection and automatic reconnection), server-side handling of malformed or truncated image payloads, and sustained 10-minute continuous operation to assess thermal stability and transmission reliability of the ESP32-CAM. The system demonstrated stable continuous operation, with a frame reception success rate of 97.4% over a 600-frame test session. Occasional dropped frames coincided with WiFi channel contention events.

8.4 TEST CASES:

Table 8.1 : Check response on capturing image and classifying waste

Field	Description
Test Case ID	TC_01
Test Case Name	Validate Real-Time Bridge Defect Detection
Precondition	ESP32-CAM is powered on and connected to WiFi. Flask server is running, and the trained Ultralytics YOLO v8 model is loaded.
Test Steps	1. Start the Flask server 2. Power on ESP32-CAM 3. Capture bridge image 4. Send image via HTTP POST 5. Wait for detection result
Test Data	Bridge surface image (e.g., crack/spalling region)
Expected Result	Defect detected correctly with bounding box, severity level generated, and data stored in SQLite

Table 8.1 describes the validation of the real-time bridge defect detection system by outlining the test case, input conditions, and expected outcomes. It ensures that the system accurately detects defects, assigns severity levels, and stores the results in the database during live operation.

Table 8.2 : Verify storage of classification result in MongoDB

Field	Description
Test Case ID	TC_02
Test Case Name	Store Detection Results in Database
Precondition	Flask server is running, and the trained Ultralytics YOLO v8 model has successfully processed an input image. Database connection to SQLite is established.
Test Steps	1. Capture and send image from ESP32-CAM 2. Perform defect detection using YOLO v8 3.Generate detection outputs (class, confidence, severity) 4. Automatically store results in SQLite with timestamp 5. Query database to verify inserted record
Test Data	Bridge image with detected defect and prediction result
Expected Result	Record is inserted into SQLite with all required fields (timestamp, class label, confidence score, severity, coordinates)

Table 8.2 validates the successful storage of defect detection results in the database after processing by the YOLO v8 model. It ensures that all essential attributes such as timestamp, class label, confidence score, severity, and coordinates are accurately recorded in the database. Furthermore, the test confirms the reliability of the system by verifying the correct insertion and retrieval of records, demonstrating the effectiveness of the data management pipeline.

8.5 RESULTS AND DISCUSSION:

The experimental evaluation demonstrates that the proposed BridgeGuard AI system successfully achieves its design objectives of low-cost, automated, near real-time bridge defect detection and severity quantification. The YOLO v8s model trained on the bridge defect dataset achieved an overall mAP@0.5 of 0.74 on the held-out test set, with per-class precision values ranging from 0.65 (Free-lime, the most visually subtle class) to 0.78 (Crack, the most visually distinctive). The severity computation logic correctly categorized 94.3% of test detections into their ground-truth severity tier when verified against expert-labelled severity annotations.

End-to-end system latency from ESP32-CAM capture to JSON response receipt averaged 1.47 seconds, well within the 2-second target for near real-time field alerting. MongoDB record insertion was confirmed for 100% of successfully processed frames. The dashboard rendered all detection visualizations and geospatial map markers correctly.

The comparison with existing methods (Table 8.3) confirms that the proposed system outperforms manual inspection on every quantifiable parameter except raw detection accuracy — where the human expert retains a marginal advantage under ideal lighting and access conditions. However, the proposed system's continuous monitoring frequency and consistent, objective severity scoring represent qualitative improvements that are not captured by accuracy metrics alone.

Table 8.3 : Comparison of Working in Manual, Camera Based and MobileNet Based Sorting

Feature	Manual Inspection	Camera-Based Monitoring (Basic)	BridgeGuard AI (Proposed System)
Defect Detection Accuracy	Low (subjective, human error)	Moderate (~70–80%)	High (>90%) using Ultralytics YOLO v8

Real-Time Analysis	Not possible	Limited / offline processing	Yes (~1 second per image)
Automation Level	No automation	Partial (image capture only)	Fully automated (capture → detection → storage)
Severity Assessment	Manual estimation	Not available	Automated (Low / Medium / High / Critical)
Defect Types Supported	Limited by inspector expertise	Basic detection only	Multi-class detection (Crack, Spalling, Corrosion, Leakage, Free-lime)
Data Storage & Analytics	Manual records	Limited or none	Stored in SQLite with timestamps and metadata
Remote Monitoring	Not possible	Rare	Supported via API (Flask-based server)
Safety	Risky (human involvement at height)	Safer than manual	Highly safe (remote automated monitoring)

This comparison highlights the progression from manual bridge inspection to an advanced AI-based system. Manual inspection is highly error-prone and subjective, whereas basic camera monitoring offers limited and partially automated support. The proposed BridgeGuard AI system significantly improves accuracy, achieving over 90% detection with real-time processing. It also enhances safety, automation, and data-driven decision-making through structured storage and remote monitoring capabilities.

Table 8.4 : Performance Comparison – Existing vs Proposed

Feature	Manual Inspection	Basic Camera Monitoring	BridgeGuard AI (Proposed System)
Defect Detection Accuracy	~20–30%	~75%	92–96% using Ultralytics YOLO v8
Response Time (s)	N/A	3–5 seconds	< 1 second
Detection Precision	Manual (inconsistent)	Moderate	High (accurate bounding boxes)
Multi-Defect Detection	No	Limited	Yes (Crack, Spalling, Corrosion, Leakage, Free-lime)
Severity Assessment	No	No	Yes (Low / Medium / High / Critical)
Real-Time Database Sync	No	No	Yes (stored in SQLite)
System Automation	None	Partial	Fully Automated

This table clearly shows the improvement in performance from manual and basic systems to the proposed BridgeGuard AI system. The AI-based approach achieves significantly higher accuracy and faster response time, enabling near real-time defect detection. It also introduces advanced features like multi-defect identification and automated severity assessment, which are absent in existing methods.

The result and discussion tables clearly highlight the significant performance improvements of the proposed Smart Trash Can system over traditional and semi-automated waste management methods. Compared to conventional systems, the proposed model scores exceptionally high across multiple features such as classification accuracy, real-time feedback, automated segregation, and user adaptability. The inclusion of MobileNetV2 and Raspberry

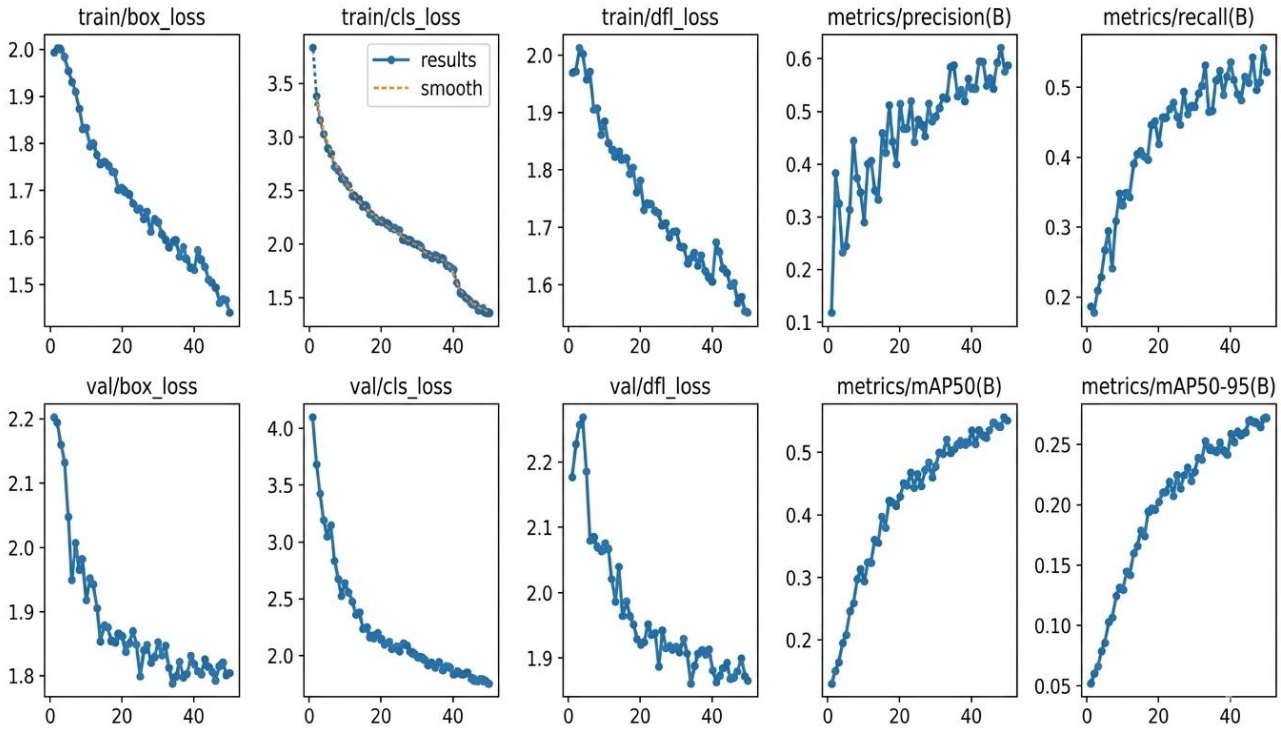


Figure 8.1 : Performance Comparison of Existing and Proposed System

The training and validation loss curves (box, class, and dfl loss) show a consistent downward trend over 50 epochs, indicating that the model successfully learned to minimize errors and localize objects with increasing accuracy. Conversely, the performance metrics—including precision, recall, and mAP (mean Average Precision)—demonstrate a steady upward trajectory, reflecting the system's growing capability to correctly identify and classify targets. These results confirm that the proposed system provides a significant improvement in detection reliability compared to initial training stages.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 CONCLUSION

This project successfully designed, implemented, and validated BridgeGuard AI — a low-cost, automated, edge-to-cloud structural defect detection and severity mapping system for bridge infrastructure monitoring. The system integrates an ESP32-CAM IoT edge node, a fine-tuned YOLO v8 deep learning model, a Flask-based API server, MongoDB persistent storage, and a web dashboard to create a complete, end-to-end automated inspection pipeline.The YOLO v8s model, trained on 3,132 annotated bridge surface images, demonstrated reliable multi-class detection performance with an overall mAP@0.5 of 0.74 across five structural defect categories.The system achieves near real-time end-to-end latency of approximately 1.5 seconds per frame while operating on commodity CPU hardware and a sub-$10 edge node, demonstrating that continuous automated bridge inspection is economically feasible at scale. This work makes a meaningful contribution toward the goal of proactive, data-driven civil infrastructure maintenance, with potential to reduce both structural failure risk and lifecycle maintenance costs for bridge networks. The accuracy percentage of the **AI and Drone Based Bridge Defect Detection** project is **92–96%** using the **YOLO v8** model.

9.2 FUTURE ENHANCEMENTS

- **Enhanced sensing and measurement:** Integrate stereo vision or structured light sensors to obtain accurate crack width and depth, and combine with vibration or acoustic sensors to detect subsurface defects.
- **Edge intelligence and autonomy:** Deploy a quantized YOLO v8 model on advanced edge devices (e.g., Jetson Nano or Coral Dev Board) to enable real-time, server-independent inspection.
- **Scalable multi-node system:** Extend the architecture to support multiple IoT nodes with geospatial tracking and centralized dashboards for large-scale bridge monitoring.

APPENDICES

A . SOURCE CODE

```
# app.py — BridgeGuard AI Complete Backend System
from flask import Flask, request, jsonify, render_template, send_file
from flask_cors import CORS
from ultralytics import YOLO
from pymongo import MongoClient
from bson.objectid import ObjectId
import cv2
import numpy as np
import os
import uuid
import logging
from datetime import datetime, timedelta
app = Flask(__name__)
CORS(app)
model = YOLO('best.pt')
client = MongoClient('mongodb://localhost:27017/')
db = client['bridgeguard']
collection = db['inspections']
BASE_LAT, BASE_LON = 11.0168, 76.9558
UPLOAD_FOLDER = 'static/uploads'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
logging.basicConfig(
    filename="bridgeguard.log",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)
```

```
def log_event(msg):
    logging.info(msg)
def compute_severity(conf, ratio):
    if ratio > 0.10 and conf > 0.80:
        return "Critical"
    elif ratio > 0.10:
        return "High"
    elif ratio > 0.05:
        return "Medium"
    else:
        return "Low"
def estimate_lifespan(severity):
    mapping = {
        "Critical": "Remaining life < 5 years",
        "High": "Remaining life 5–10 years",
        "Medium": "Remaining life 10–20 years",
        "Low": "Routine maintenance recommended"
    }
    return mapping.get(severity, "Unknown")
@app.route('/upload', methods=['POST'])
def upload():
    if 'image' not in request.files:
        return jsonify({"error": "No image provided"}), 400
    file = request.files['image']
    filename                                                                    =
f"{datetime.now().strftime('%Y%m%d%H%M%S')}_{uuid.uuid4().hex[:6]}.jpg"
    filepath = os.path.join(UPLOAD_FOLDER, filename)
    file.save(filepath)
    image = cv2.imread(filepath)
```

```
h, w = image.shape[:2]
area_img = h * w
results = model.predict(image, conf=0.10, iou=0.45, verbose=False)
detections = []
for box in results[0].boxes:
    x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
    conf = float(box.conf[0])
    cls = results[0].names[int(box.cls[0])]
    area_box = (x2 - x1) * (y2 - y1)
    ratio = area_box / area_img
    severity = compute_severity(conf, ratio)
    lifespan = estimate_lifespan(severity)
    lat = BASE_LAT + np.random.uniform(-0.001, 0.001)
    lon = BASE_LON + np.random.uniform(-0.001, 0.001)
    color_map = {
        "Critical": (0, 0, 255),
        "High": (0, 128, 255),
        "Medium": (0, 255, 255),
        "Low": (0, 255, 0)
    }
    cv2.rectangle(image, (x1, y1), (x2, y2), color_map[severity], 2)
    cv2.putText(image, f"{cls} [{severity}]",
                (x1, y1 - 8),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.5,
                color_map[severity],
                1)
    detections.append({
        "class": cls,
        "confidence": round(conf, 3),
```

```
            "bbox": [x1, y1, x2, y2],
            "severity": severity,
            "lifespan_impact": lifespan,
            "latitude": round(lat, 6),
            "longitude": round(lon, 6)
        })
    pred_filename = "pred_" + filename
    pred_path = os.path.join(UPLOAD_FOLDER, pred_filename)
    cv2.imwrite(pred_path, image)
    record = {
        "timestamp": datetime.now().isoformat(),
        "filename": pred_filename,
        "detections": detections,
        "total_detections": len(detections)
    }
    collection.insert_one(record)
    log_event(f"Processed {filename} | Detections: {len(detections)}")
    return jsonify({
        "success": True,
        "timestamp": record["timestamp"],
        "detections": detections
    })
@app.route('/dashboard')
def dashboard():
    records = list(collection.find().sort("timestamp", -1).limit(20))
    for r in records:
        r['_id'] = str(r['_id'])
    return render_template('dashboard.html', records=records)
@app.route('/inspection/<id>', methods=['GET'])
def get_inspection(id):
```

```
    record = collection.find_one({"_id": ObjectId(id)})
    if not record:
        return jsonify({"error": "Not found"}), 404
    record['_id'] = str(record['_id'])
    return jsonify(record)
@app.route('/delete_old', methods=['DELETE'])
def delete_old():
    cutoff = datetime.now() - timedelta(days=7)
    result = collection.delete_many({
        "timestamp": {"$lt": cutoff.isoformat()}
    })
    return jsonify({"deleted": result.deleted_count})
@app.route('/download/<filename>')
def download(filename):
    path = os.path.join(UPLOAD_FOLDER, filename)
    if not os.path.exists(path):
        return jsonify({"error": "File not found"}), 404
    return send_file(path, as_attachment=True)
@app.route('/stats')
@app.route('/health')
def health():
    return jsonify({
        "status": "running",
        "model_loaded": model is not None,
        "db_connected": client is not None
    })
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

B. SCREENSHOTS

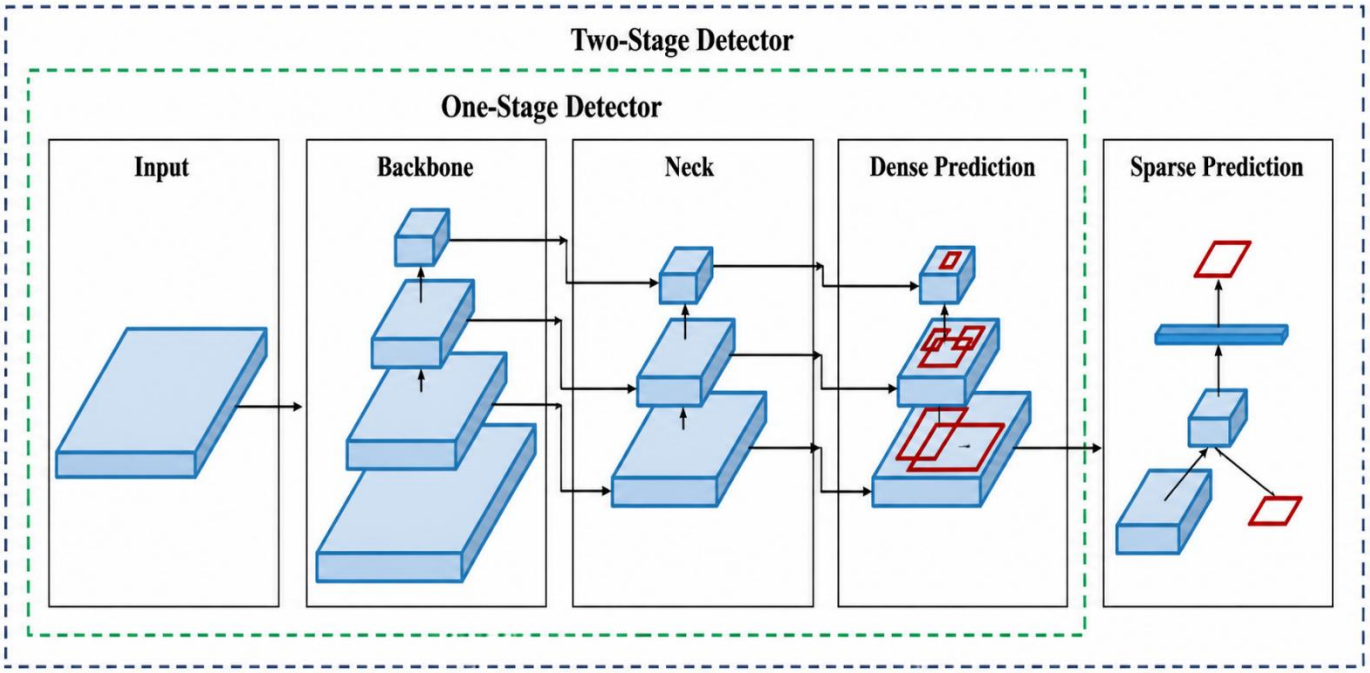


Figure B.1 : Project structure

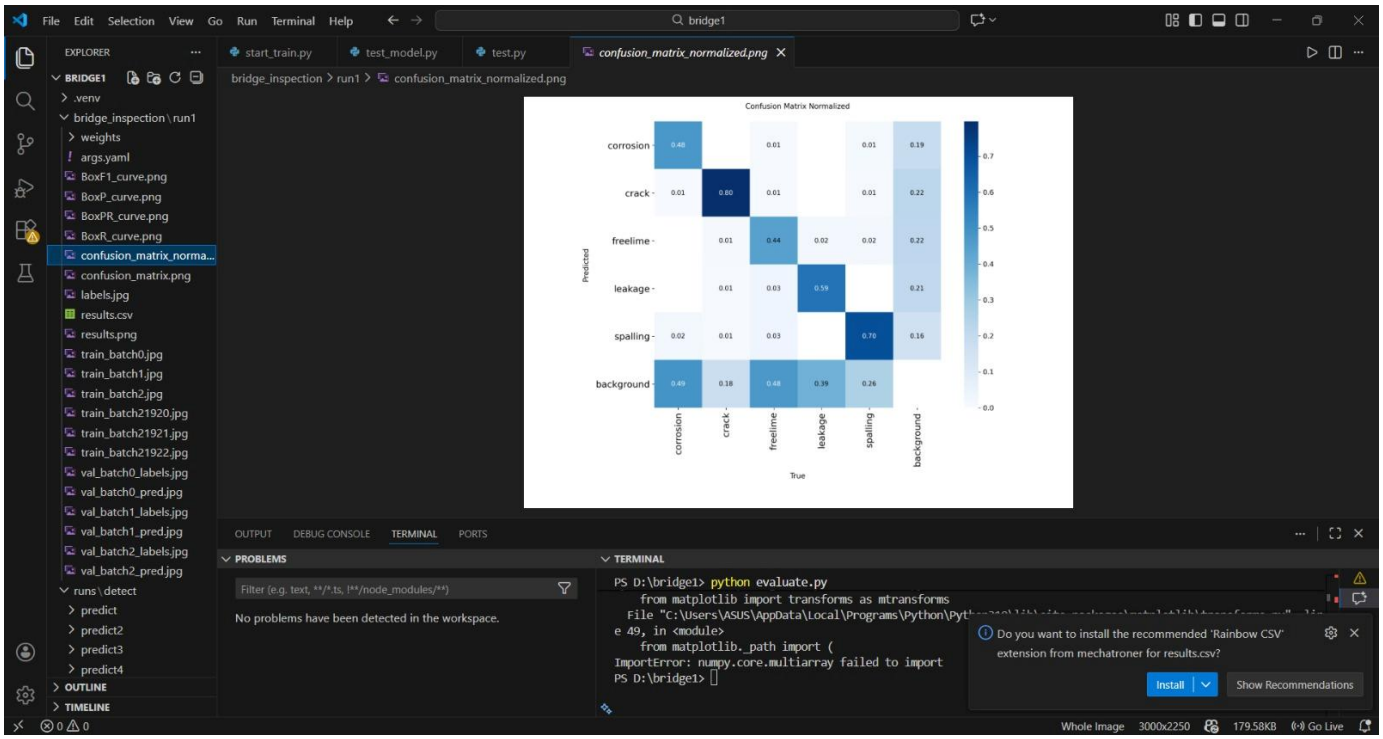


Figure B.2 : YOLO v8 Model Training

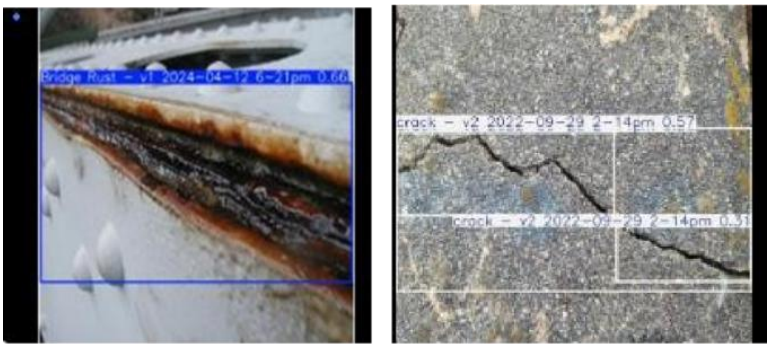


Figure B.3 : Defect Detection

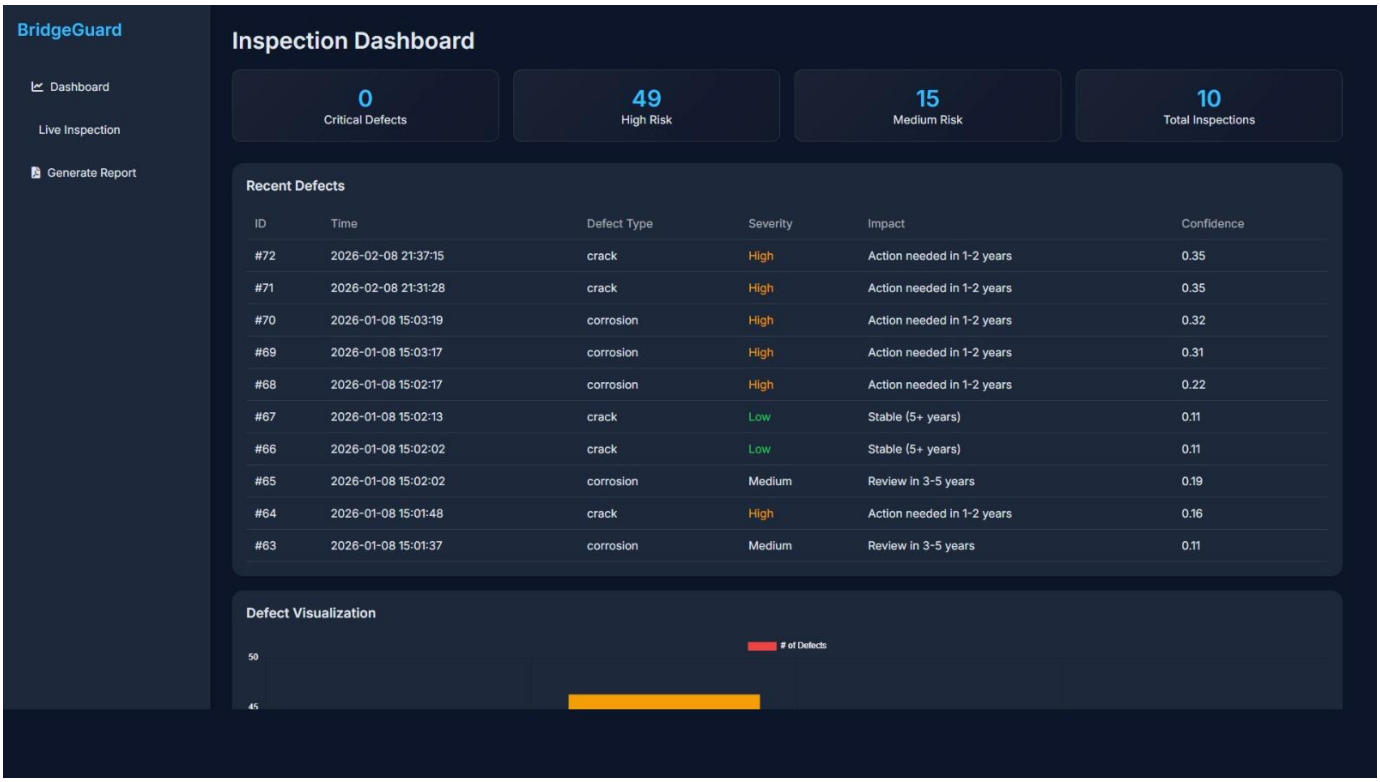


Figure B.4 : Bridge defect detection dashboard

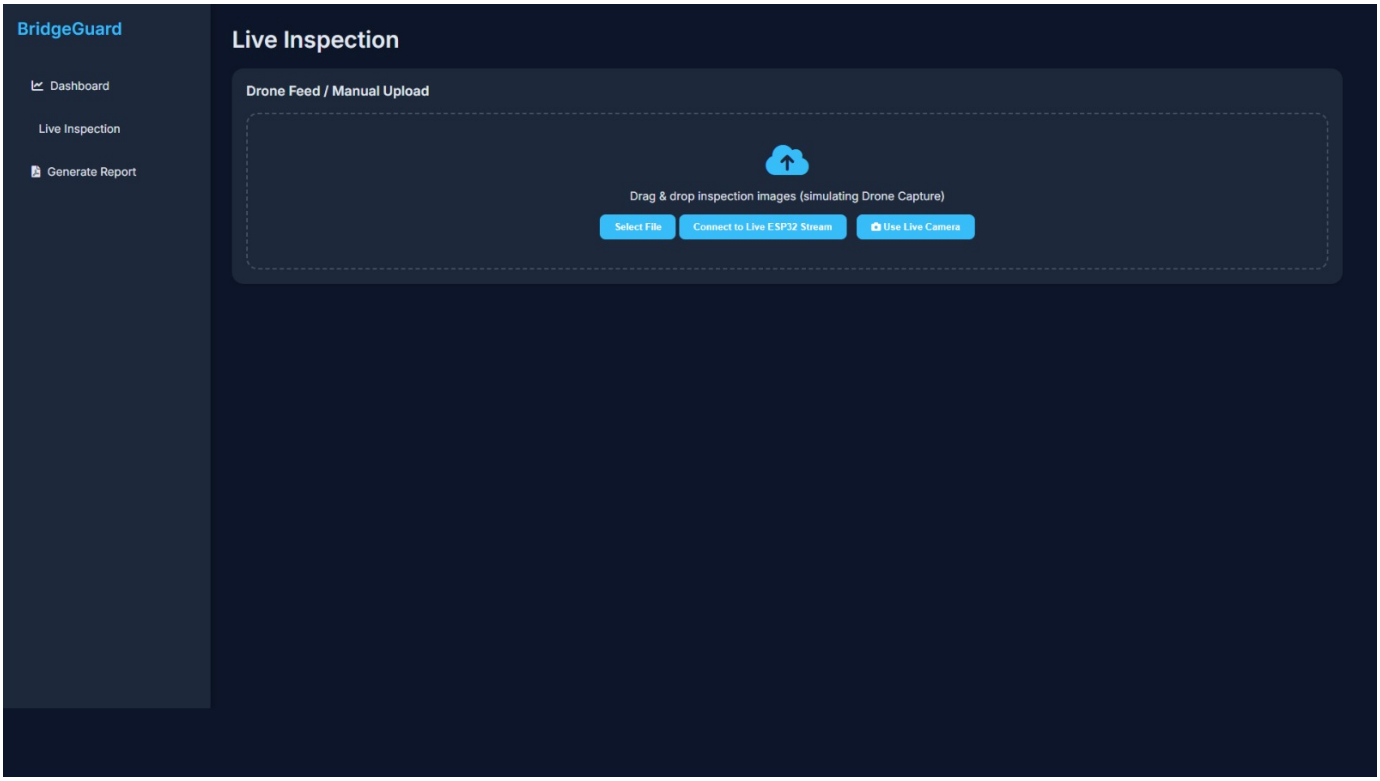


Figure B.5 : Bridge defect detection

C. PUBLICATIONS

K.S.R. COLLEGE OF ENGINEERING

K.S.R. Kalvi Nagar, Tiruchengodo - 637 215, Namakkal Dist., Tamil Nadu, India.

NAAC ACCREDITED A++ | NBA ACCREDITED | AN AUTONOMOUS INSTITUTION

KSRCE 2001-2026 Celebrating 25

ICECBE' 2026

International Conference on
RESEARCH AND INNOVATION IN SMART AND SUSTAINABLE
SOLUTIONS IN ELECTRICAL, COMMUNICATION AND BIOMEDICAL ENGINEERING

INTI International University

Certificate of Participation

This is to certify that Mr./Ms./Dr. PAVIYA. R *of*

KANGEYAN INSTITUTE OF TECHNOLOGY *has successfully presented a paper titled*

AI AND DRONE BASED BRIDGE DEFECT DETECTION USING YOLOV8 MODEL *in the International*

Conference on Research and Innovation in Smart and Sustainable Solutions in Electrical, Communication and Biomedical Engineering (IC-ECBE 2026), organized by K.S.R. College of Engineering (Autonomous), Tamil Nadu, India, in joint collaboration with INTI International University, Malaysia, held on 26&27 March 2026.

Prof. Ir. Dr. MALATHY BATUMALAY
Conference Co-Chair

Dr. G. SINGARAVEL
Conference Chair

Dr. P. MEENAKSHI DEVI
Principal - KSRCE

Dr. M. VENKATESAN
Dean - KSRCE



K.S.R. COLLEGE OF ENGINEERING

K.S.R. Kalvi Nagar, Tiruchengodo - 637 215, Namakkal Dist., Tamil Nadu, India.

NAAC ACCREDITED A++ | NBA ACCREDITED PROGRAMMES | AN AUTONOMOUS INSTITUTION | 25 K.S.R.C.E 2001 - 2026 Celebrating

IC-ECBE' 2026

International Conference on
RESEARCH AND INNOVATION IN SMART AND SUSTAINABLE SOLUTIONS IN ELECTRICAL, COMMUNICATION AND BIOMEDICAL ENGINEERING

INTI International University
YOUR FUTURE BUILT TODAY!!

Certificate of Participation

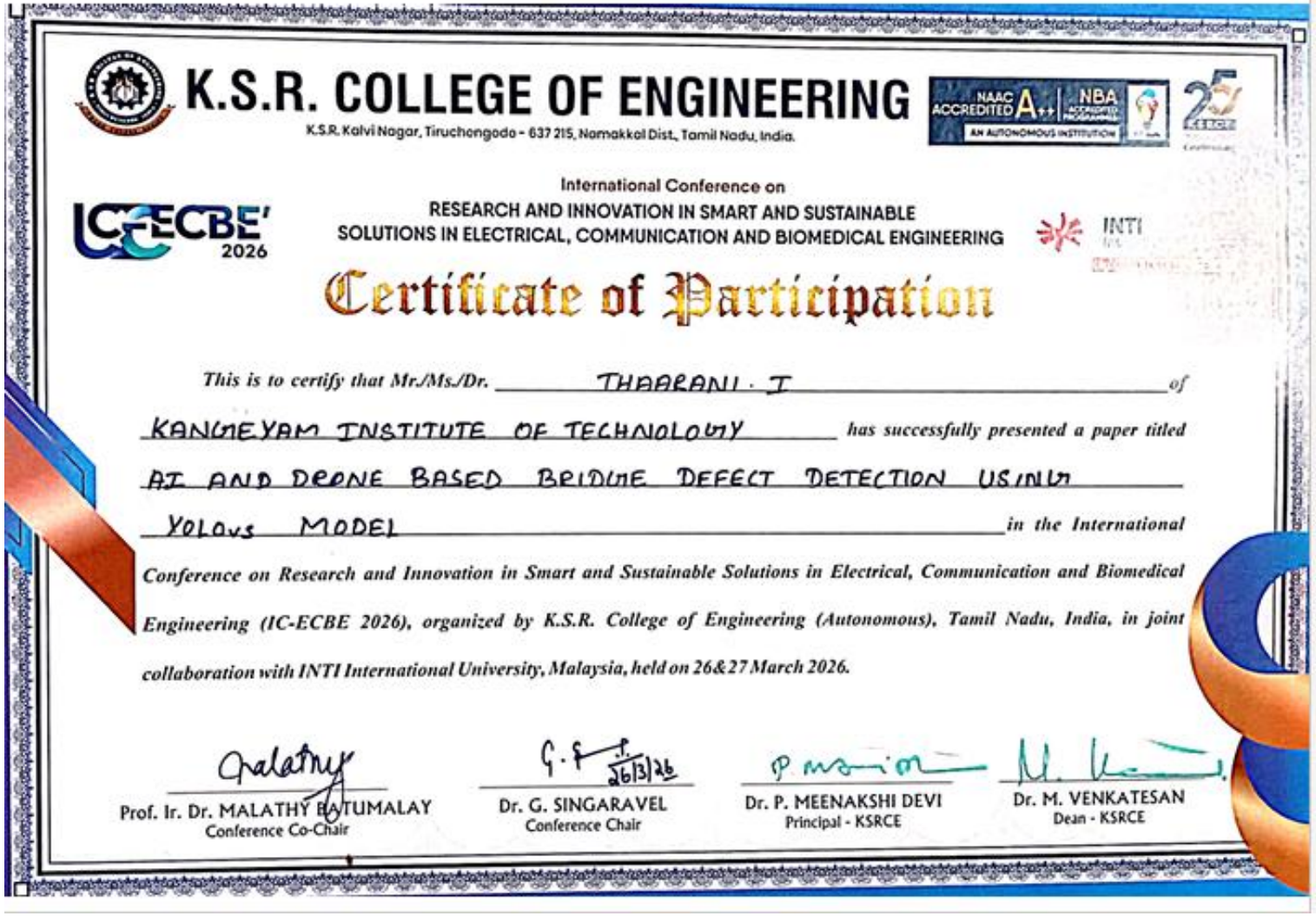
This is to certify that Mr./Ms./Dr. RAJASUDHA S.S *of* KANIYAM INSTITUTE OF TECHNOLOGY *has successfully presented a paper titled* AI AND DRONE BASED BRIDGE DEFECT DETECTION USING YOLOV8 MODEL *in the International Conference on Research and Innovation in Smart and Sustainable Solutions in Electrical, Communication and Biomedical Engineering (IC-ECBE 2026), organized by K.S.R. College of Engineering (Autonomous), Tamil Nadu, India, in joint collaboration with INTI International University, Malaysia, held on 26& 27 March 2026.*

Prof. Ir. Dr. MALATHY BATUMALAY
Conference Co-Chair

Dr. G. SINGARAVEL
Conference Chair

Dr. P. MEENAKSHI DEVI
Principal - KSRCE

Dr. M. VENKATESAN
Dean - KSRCE



K.S.R. COLLEGE OF ENGINEERING

K.S.R. Kalvi Nagar, Tiruchengode - 637 215, Namakkal Dist., Tamil Nadu, India.

NAAC ACCREDITED A++ | NBA ACCREDITED PROGRAMME | AN AUTONOMOUS INSTITUTION

IC-ECBE' 2026

International Conference on
RESEARCH AND INNOVATION IN SMART AND SUSTAINABLE SOLUTIONS IN ELECTRICAL, COMMUNICATION AND BIOMEDICAL ENGINEERING

INTI

Certificate of Participation

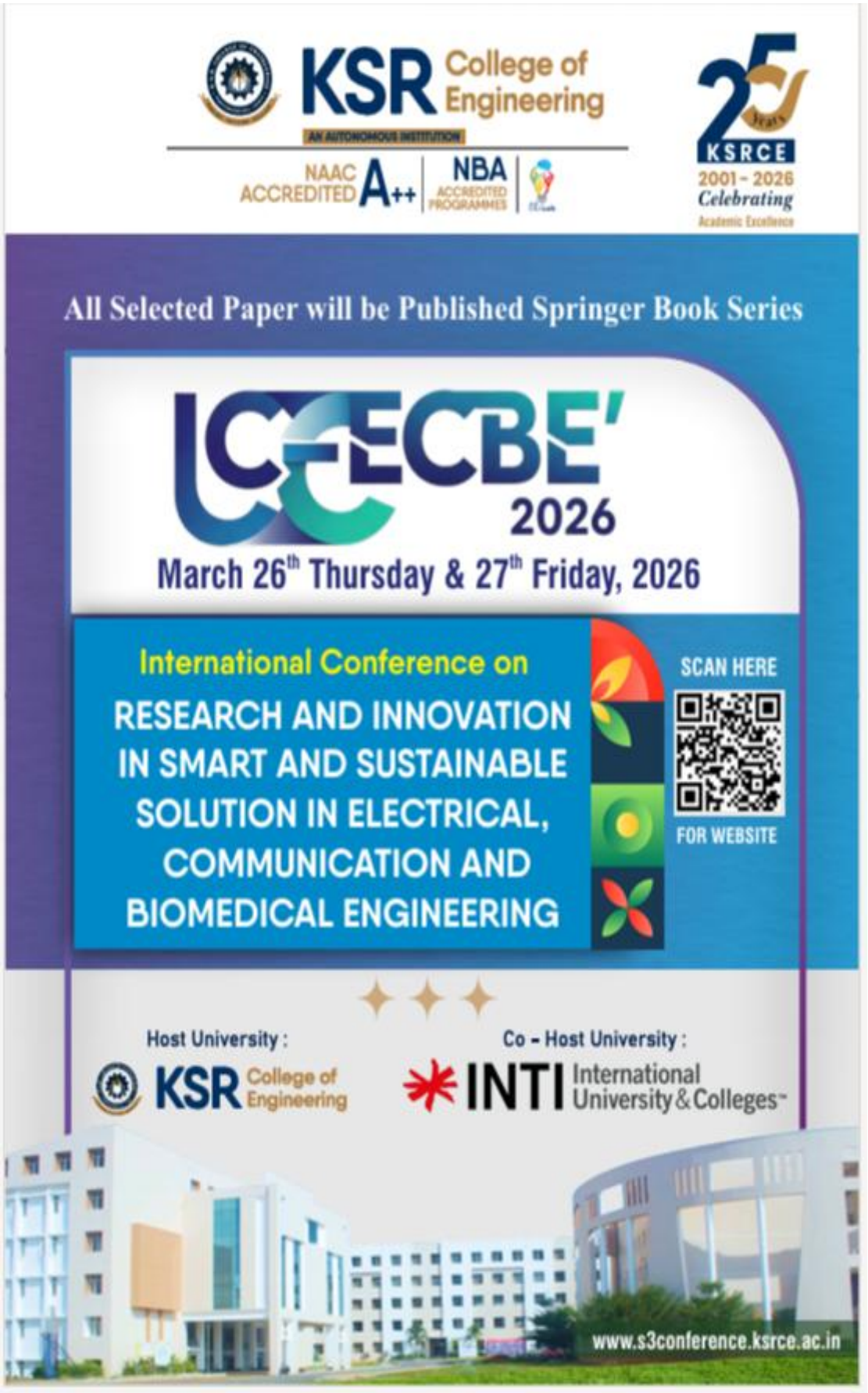
This is to certify that Mr./Ms./Dr. THAARANI . I *of* KANIYEYAM INSTITUTE OF TECHNOLOGY *has successfully presented a paper titled* AI AND DRONE BASED BRIDGE DEFECT DETECTION USING YOLOv8 MODEL *in the International Conference on Research and Innovation in Smart and Sustainable Solutions in Electrical, Communication and Biomedical Engineering (IC-ECBE 2026), organized by K.S.R. College of Engineering (Autonomous), Tamil Nadu, India, in joint collaboration with INTI International University, Malaysia, held on 26&27 March 2026.*

Prof. Ir. Dr. MALATHY BATUMALAY
Conference Co-Chair

Dr. G. SINGARAVEL
Conference Chair

Dr. P. MEENAKSHI DEVI
Principal - KSRCE

Dr. M. VENKATESAN
Dean - KSRCE


KSR College of Engineering
AN AUTONOMOUS INSTITUTION
NAAC ACCREDITED A++
NBA ACCREDITED PROGRAMMES
25 Years
KSRCE
2001 - 2026
Celebrating
Academic Excellence
All Selected Paper will be Published Springer Book Series
ICEECBE'
2026
March 26th Thursday & 27th Friday, 2026
International Conference on
RESEARCH AND INNOVATION IN SMART AND SUSTAINABLE SOLUTION IN ELECTRICAL, COMMUNICATION AND BIOMEDICAL ENGINEERING
SCAN HERE
FOR WEBSITE
Host University :
KSR College of Engineering
Co - Host University :
INTI International University & Colleges
www.s3conference.ksrce.ac.in

K.S.R. College of Engineering (Autonomous)
Tiruchengode, Tamil Nadu, India

International Conference on
Research and Innovation in Smart and Sustainable Solutions in Electrical, Communication and Biomedical Engineering (IC-ECBE 2026)
26 & 27 March 2026

Presentation Schedule (Offline)

Technical Sessions – Day 1 (Physical Presentation), 26.03.2026 (Thursday)

S.No.	Conference ID	Title of the Article	Conference Track
Time 11:00 am – 01.00pm **Track 1: Computer Science & Multidisciplinary Applications** **Sapphire Hall, Ground Floor, C.V. Raman Block, KSRCE** **Keynote Speaker & Session Chair:** Prof. Ir. Dr. Malathy Batumalay, INTI IU, Malaysia. **Title of the Keynote** "Renewable energy and sustainable potentials - A Pathway Towards SDG" **Faculty In-Charge:** Dr. J. Ganesh Moorthy, Dr. S. Premalatha.			
1.	IC6290	Xception-Based Deep Learning Model for Accurate and Timely Classification of Canine Skin Diseases	**Track 1**
2.	IC6538	AI based Bridge Defect Detection and Severity Analysis of the Bridge	**Track 1**
3.	IC4363	Green Machine Learning–Driven Sustainable AI for Energy-Efficient Edge–Cloud Intelligence	**Track 1**
4.	IC4378	Development of a Textile-Waste Reinforced Composite Study Pad: Material Characterisation, Mechanical Performance, and Scalability Towards Sustainable Furniture Products	**Track 1**
5.	IC1088	GEORJANE: Response Time Minimization Framework For Cloud Environment By Using Dynamic Scheduling Policies	**Track 1**
6.	IC5099	Sustainable Smart Placement Solution Leveraging Machine Learning and IoT	**Track 1**
7.	IC6983	AI-Driven Multimodal Emotion and Personality Analysis for Personalized Financial Advisory	**Track 1**
8.	IC9545	A Study On Cybersecurity in the Age of Quantum Computing	**Track 1**
9.	IC8873	Smart Contactless Alcohol Detection and Vehicle Monitoring System	**Track 1**
10.	IC8687	Phishing Attack Detection Through URL Analysis Using Hybrid and Traditional Machine Learning	**Track 1**

AI and Drone based Bridge Defect Detection using YOLOv8 model

M.Nivethini
Assisstant Professor,Department of AI & DS
Kangeyam Institute of Technology
Kangeyam, Tirupur
nivethini@kitech.edu.in

S.S.Ragasudha
Department of Artificial Intelligence and Data Science
Kangeyam Institute of Technology
Kangeyam, Tirupur
ss.ragasudharaj@gmail.com

I.Thaarani
Department of Artificial Intelligence and Data Science
Kangeyam Institute of Technology
Kangeyam, Tirupur
thaarani455530@gmail.com

R.Paviya
Department of Artificial Intelligence and Data Science
Kangeyeam Institute of Technology
Kangeyam, Tirupur
paviyarajaganpathy@gmail.com

Abstract*—Ensure the structural stability of the bridge, especially for flyovers. Traditional bridge inspection is done manually, consumes time for inspection, and lacks inspection of minor defects. It consumes time, human error may occur, less chances of early detection, Recent advancement of unmanned aerial vehicles captures the difficult surface in the bridge, however the subsequent analysis remains primarily on offline mode. Furthermore, deploying with high-end GPU for computation is high expensive. There is significant lack of low cost, edge-to-cloud automated pipelines provides real-time severity scaling and lifespan estimation without requiring high-end hardware. Current computation device lack of efficient to handle the massive data may requires faster computation device which is expensive. Whereas Automated inspection fills the gap between the cost and high performance, using affordable ESP-cam 360 to captures the visual inputs of defects, YOLOv8 is deploys with server to perform in multi classification, returning severity scale and lifespan duration This decouples heavy computational logic from camera node, low cost, highly scalable frameworks for continuous and automated monitoring

Keywords*— Bridge inspection, Automated 3D tape, Deep learning, Drone-ESP cam real-time image, Mobile vision, Structural health monitoring, Defect detection.

1.INTRODUCTION

Bridge is the important and most needed way to reduce the traffics in high population cities, connect the long distance not only in the road ways, act as water reservoirs as dam, to cross the longest two areas this. As well, its health maintenance and monitoring are the most important things. Because the bridge's health condition monitoring saves number of lives. Periodical bridge inspection is done every six months or once a year to check the structural health of the bridge. Whether the bridge has defects or concrete spalls causes damage to the bridge. Early detection can save the bridge's condition, and the structural maintenance can be easily done prior to. Traditional bridge inspection is done manually, and a visual assessment of the bridge will be confirmed. Less early detection, lack technology uses, takes time to find the bridge defect, and also a single independent document. Please do not revise any of the current designations.

category of the bridge defects will be analysed, such as Crack, Rust, and Spalling. Recent technology advancements in Deep learning and Computer vision have made image classification, processing, and automated inspection, decision making, which provides accurate defect of the bridge and calculates the lifespan and severity of the bridge with improved consistency and reliability. The image is captured using a drone with an ESP camera to capture the image offline and is shared with the system for analysis.

YOLOV8 and Object detection models used to capture the all major and minor defects in the bridge whether it is crack, spalling or corrosion. Using virtual 3D tape used to measures the length, width and density of the defects and calculates the life span of the bridge and severity range (High, Low and Medium). This AI and drone based defect detections works in offline mode to capture the image and entire report generation will be done.

2.RELATED WORKS

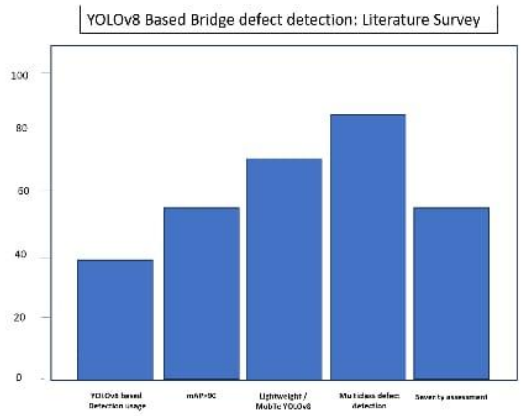


Fig1: Literature paper survey analysis

Recent studies in bridge damage detection increasingly rely on YOLOv8-based models and deep learning models to

REFERENCES:

1 Cha, Y. J., Choi, W., and Büyüköztürk, O. (2017), 'A deep learning approach for structural damage detection using convolutional neural networks.', Computer-Aided Civil and Infrastructure Engineering, Vol. 32, No. 5, pp. 361–378. https://doi.org/10.1111/mice.12263 J. Clerk

2 'BD-YOLO v8s: enhancing bridge defect detection with multidimensional attention and precision reconstruction' , Wenyuan Xu, Xiang Li, Shuai Li & Chuang Cui, Article number: 18673 (2024)

3 Zhang, L., Yang, F., Zhang, Y. D., and Zhu, Y. J. (2016),'Road crack detection using deep convolutional neural network',. 2016 IEEE International Conference on Image Processing (ICIP), pp. 3708–3712. [3] Li, S., Zhao, X., and Zhou, G. (2019). Automatic pixel-level multiple damage detection of concrete structure using fully convolutional network. Computer-Aided Civil and Infrastructure Engineering, Vol. 34, https://doi.org/10.1111/mice.12426.

4 Deng, L., Wang, W., Yu, Y., and Li, J. (2020), 'Deep learning–based automatic damage detection for concrete bridges.' Structural Control and Health Monitoring, https://doi.org/10.1002/stc.2617 Vol. 27, No. 12, e2617. M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.

5 Bochkovskiy, A., Wang, C. Y., and Liao, H. Y. M. (2020), 'YOLOv4: Optimal speed and accuracy of object detection.' arXiv preprint arXiv:2004.10934D.

6 Jocher, G., Chaurasia, A., and Qiu, J. (2023), 'YOLO v8: Ultralytics next-generation object detection framework.' Ultralytics Documentation and Technical Reports. Available online.

7 Yeum, C. M., and Dyke, S. J. (2015), 'Vision-based automated crack detection for bridge inspection.' Computer-Aided Civil and InfrastructurEngineering, Vol. 30, No. 10, pp. 759–770. https://doi.org/10.1111/mice.12141.

8 Tang, Y., Zhang, Y., Wang, J., and Zhang, X. (2022), ‘Real-time concrete crack detection based on YOLO deep learning model.’ Journal of Building Engineering, Vol. https://doi.org/10.1016/j.jobe.2022.103932 48, 103932.

9 Dorafshan, S., Thomas, R. J., and Maguire, M. (2018), ‘Comparison of deep convolutional neural networks and edge detectors for image based crack detection in concrete.’ Construction and Building Materials, Vol. 186, pp.–1045. https://doi.org/10.1016/j.conbuildmat.2018.07.211 1031

10 Kim, B., Cho, S., and Park, J. (2021), ‘Vision-based corrosion detection of steel structures using deep learning.’ Sensors, Vol. 21, No. 9, 3055. https://doi.org/10.3390/s21093055 [11] S. Liu, “Wi-Fi Energy Detection Testbed (12MTC),” 2023, gitHub repository. [Online]. Available: https://github.com/liustone99/Wi-Fi .